# TIS Project Reference Booklet

Documentation version: 3.0

Generated: 2026-06-27 11:09 Arab Standard Time

Branch: dev

Git commit SHA: 1a53a89

Source of truth: Markdown files under docs/ are authoritative. This PDF is a generated snapshot and must not be edited manually.

## Source Documents Included

- docs/README.md
- docs/AI_PROJECT_CONTEXT.md
- docs/DOCUMENTATION_UPDATE_POLICY.md
- docs/TIS_MASTER_CONTEXT.md
- docs/PROJECT_STATE.md
- docs/CHANGE_HISTORY.md
- docs/adr/README.md
- docs/engineering/README.md
- docs/engineering/TIS_MODULE_MAP.md
- docs/engineering/REPOSITORY_ARCHITECTURE.md
- docs/engineering/USER_AND_SYSTEM_FLOWS.md
- docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
- docs/engineering/DEVELOPMENT_STANDARDS.md
- docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
- docs/engineering/PRODUCT_ROADMAP.md
- docs/engineering/REJECTED_DECISIONS.md
- docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
- docs/engineering/AI_OPTIMIZATION_GUIDE.md
- docs/engineering/PROJECT_GOVERNANCE.md
- docs/engineering/KNOWLEDGE_LIFECYCLE.md
- docs/engineering/DOCUMENTATION_AUTOMATION.md
- docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
- docs/engineering/SELF_EVOLVING_WORKFLOW.md
- docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md

- docs/engineering/AI_CODING_WORKFLOW.md
- docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
- docs/adr/0001-separate-nextjs-landing-website.md
- docs/adr/0002-separate-saas-identity-and-operational-users.md
- docs/adr/0003-paddle-payment-architecture.md
- docs/adr/0004-webhook-only-payment-confirmation.md
- docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
- docs/adr/0006-documentation-as-source-knowledge-management-system.md
- docs/adr/0007-landing-page-visual-system-strategy.md
- docs/history/academic-calendar/README.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md
- docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md
- docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md
- docs/history/engineering-handbook/README.md
- docs/history/landing-page/README.md
- docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md
- docs/history/platform-knowledge/README.md
- docs/history/provisioning/2026-06-26-kms-foundation.md
- docs/history/provisioning/README.md
- docs/history/README.md
- docs/history/saas-onboarding/README.md
- docs/history/subscriptions/README.md
- docs/history/workforce-planning/README.md
- docs/marketing/landing_page_source_of_truth.md
- docs/marketing/tis_landing_page_master_content.md
- docs/location-data-roadmap.md

# docs/README.md

# TIS Documentation

This folder is the source of truth for the TIS Knowledge Management System (KMS).

Markdown files are authoritative. The PDF booklet is a generated snapshot and must never be edited manually.

## First Read For AI Coding Conversations

For any new Codex or ChatGPT coding conversation, load this file first:

1. `docs/AI_PROJECT_CONTEXT.md`

Then load:

1. `docs/TIS_MASTER_CONTEXT.md`
1. `docs/PROJECT_STATE.md`
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`
1. relevant ADRs under `docs/adr/`
1. relevant module history under `docs/history/`
1. engineering handbook docs under `docs/engineering/`

If the task touches the public website, also read:

1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/marketing/tis_landing_page_master_content.md`
1. `docs/adr/0001-separate-nextjs-landing-website.md`
1. `docs/adr/0007-landing-page-visual-system-strategy.md`

## Core Documents

1. `docs/AI_PROJECT_CONTEXT.md`: compact onboarding file for future Codex and ChatGPT conversations.
1. `docs/TIS_MASTER_CONTEXT.md`: long-term product, architecture, SaaS, workflow, roadmap, and critical-rules source of truth.
1. `docs/PROJECT_STATE.md`: living status file for branch strategy, priority, milestones, known issues, and next work.
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`: non-negotiable KMS and Knowledge Impact Assessment policy.
1. `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.

## Engineering Handbook

1. `docs/engineering/README.md`: engineering onboarding order and handbook index.
1. `docs/engineering/TIS_MODULE_MAP.md`: complete module map with purpose, files, maturity, docs, risks, and guardrails.
1. `docs/engineering/REPOSITORY_ARCHITECTURE.md`: repository structure and ownership boundaries.

- `docs/engineering/USER_AND_SYSTEM_FLOWS.md`: public customer, SaaS identity, payment, provisioning, operational login, platform owner, KMS, and developer onboarding flows.
- `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`: conceptual data model and tenant/identity isolation rules.
- `docs/engineering/DEVELOPMENT_STANDARDS.md`: non-negotiable engineering rules.
- `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`: UI/UX principles by product surface.
- `docs/engineering/PRODUCT_ROADMAP.md`: completed, current, next, and future roadmap.
- `docs/engineering/REJECTED_DECISIONS.md`: major rejected alternatives and long-term consequences.
- `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md`: screenshot, diagram, naming, storage, and visual update standards.
- `docs/engineering/AI_OPTIMIZATION_GUIDE.md`: definitive onboarding guide for future AI assistants.
- `docs/engineering/PROJECT_GOVERNANCE.md`: governance, approvals, quality gates, and decision traceability.
- `docs/engineering/KNOWLEDGE_LIFECYCLE.md`: lifecycle from planning through production and maintenance.
- `docs/engineering/DOCUMENTATION_AUTOMATION.md`: current/future KMS automation model.
- `docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md`: formal KIA engineering standard.
- `docs/engineering/SELF_EVOLVING_WORKFLOW.md`: official workflow that keeps software and KMS synchronized.
- `docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md`: propagation map between source docs, manifest, PDF, and Knowledge Center.
- `docs/engineering/AI_CODING_WORKFLOW.md`: disciplined workflow for future AI coding assistants.
- `docs/engineering/FUTURE_AUTOMATION_ROADMAP.md`: future automation opportunities.

## Decision And History Documents

- `docs/adr/`: Architecture Decision Records for major accepted decisions.
- `docs/history/`: module-based history preserving deeper before/after context.

Current module history areas:

- `docs/history/subscriptions/`
- `docs/history/landing-page/`
- `docs/history/academic-calendar/`
- `docs/history/workforce-planning/`
- `docs/history/saas-onboarding/`
- `docs/history/provisioning/`
- `docs/history/platform-knowledge/`
- `docs/history/engineering-handbook/`

## Supporting Documents

- `docs/location-data-roadmap.md`: location data roadmap and related implementation notes.

1. `docs/marketing/landing_page_source_of_truth.md`: boundary between the public Next.js landing website and the FastAPI application portal.
1. `docs/marketing/tis_landing_page_master_content.md`: approved marketing foundation and landing page content direction.

## Generated Snapshot

Generated files:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`
1. `static/docs/docs_manifest.json`

The PDF is generated from approved Markdown docs. The manifest records the generated timestamp, documentation version, branch, commit SHA, included sources, and source hashes.

## PDF Generation

Regenerate the PDF booklet with:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

The generator uses existing `reportlab` only. It must not require LaTeX, Playwright, Chromium, network calls, or system fonts.

## Knowledge Impact Assessment

Every future implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included source docs change, regenerate the PDF.

## Phase Boundary

Phase 2A and Phase 2B establish KMS governance, ADRs, module history, AI context, and PDF/manifest generation.

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation with self-evolving documentation workflow, KIA standard, dependency map, AI coding workflow, and future automation roadmap. The Platform Owner Knowledge Center is implemented as a read-only owner utility. Do not add a regenerate button or app-side Markdown rewriting unless explicitly approved.

# docs/AI_PROJECT_CONTEXT.md

# TIS AI Project Context

This is the first file future Codex or ChatGPT coding conversations should load. It is a compact project onboarding reference; detailed source of truth remains in the other Markdown docs.

## What TIS Is

TIS is Teacher Information System, a developing SaaS academic operations platform for schools and school groups. It connects teacher information, staffing and workload planning, academic calendars, observations, branch context, SaaS onboarding, billing, provisioning, and future AI-assisted academic decision support.

Public URLs:

1. Public website: `https://tisplatform.com`
1. Application portal: `https://app.tisplatform.com`

Important routes:

1. Operational login: `/login`
1. SaaS signup: `/saas/signup`
1. SaaS login: `/saas/login`
1. SaaS account: `/saas/account`
1. Platform console: `/platform`

## Current Architecture

The operational app is a FastAPI application at the repository root.

Key files and folders:

1. `main.py`: primary FastAPI app and many route handlers.
1. `auth.py`: authentication, roles, platform identity helpers, permissions, sessions.
1. `authorization.py`: protected route rules and access-denied handling.
1. `permission_registry.py`: permission keys, groups, defaults, and developer-assignable permissions.
1. `location_service.py`: global location picker lookup/validation. Must stay memory-conscious and scoped; do not restore full unbounded dataset parsing for normal picker requests.
1. `ui_shell.py`: shared app shell/navigation/page metadata.
1. `models.py`, `database.py`, `db_migrations.py`: data model, DB setup, local schema repair/migration logic.
1. `routers/`: modular operational routes.
1. `saas/`: SaaS account, onboarding, payment, billing, and provisioning services/routes.
1. `templates/`: Jinja templates.
1. `static/`: static assets and generated documentation output.
1. `tests/`: pytest coverage.

The public marketing website is separate:

- `tis-landing-website/`
- Next.js / Node runtime
- Source of truth for public landing implementation

Legacy FastAPI landing files are not the source of truth:

- `templates/landing.html`
- `static/landing/landing.css`

## Engineering Handbook

For deeper onboarding, read:

- `docs/engineering/README.md`
- `docs/engineering/TIS_MODULE_MAP.md`
- `docs/engineering/REPOSITORY_ARCHITECTURE.md`
- `docs/engineering/USER_AND_SYSTEM_FLOWS.md`
- `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`
- `docs/engineering/DEVELOPMENT_STANDARDS.md`
- `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`
- `docs/engineering/PRODUCT_ROADMAP.md`
- `docs/engineering/REJECTED_DECISIONS.md`
- `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md`
- `docs/engineering/AI_OPTIMIZATION_GUIDE.md`
- `docs/engineering/PROJECT_GOVERNANCE.md`
- `docs/engineering/KNOWLEDGE_LIFECYCLE.md`
- `docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md`
- `docs/engineering/SELF_EVOLVING_WORKFLOW.md`
- `docs/engineering/AI_CODING_WORKFLOW.md`

These files explain module ownership, repository boundaries, end-to-end flows, and what must not be changed casually.

## Domains And Routing

The public website lives at `https://tisplatform.com`. The app portal lives at `https://app.tisplatform.com`.

SaaS account routes are under `/saas`. Platform-owner SaaS administration routes are under `/saas-admin`. Operational tenant workflows use routes such as `/dashboard`, `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, and `/observations`.

## Completed M1-M5 Milestones

M1: SaaS identity foundation and separation between platform, tenant, and SaaS account identities.

M2: SaaS onboarding flow for organization, contacts, branches, academic setup, and review.

M3: Billing and plan foundation with plan catalog, checkout, billing status, and payment service boundaries.

M4: Tenant provisioning foundation with pending organizations, provisioning jobs, retry/run actions, and platform owner oversight.

M5: Platform access and owner controls, including platform owner/developer identities, permissions, and platform console behavior.

## Current Priority

Current priority is the TIS Knowledge Management System:

1. Markdown is source of truth.
1. PDF is generated snapshot.
1. Change history preserves chronological change context.
1. ADRs preserve major decisions.
1. Module history preserves deeper area-specific evolution.
1. Platform Owner Knowledge Center is implemented as a read-only owner utility.
1. The Knowledge Center uses protected routes for PDF view/download and does not link directly to static PDF paths.
1. KMS v3.0 Phase 3A adds a true engineering handbook with module map, repository architecture, workflows, and developer onboarding.
1. KMS v3.0 Phase 3B adds database architecture, development standards, UI/UX philosophy, product roadmap, and stronger human/AI developer guidance.
1. KMS v3.0 Phase 3C adds rejected decisions, visual documentation framework, AI optimization guidance, project governance, and decision traceability.
1. KMS v3.0 Phase 3D completes KMS v1.0 lifecycle standards, dependency mapping, AI coding workflow, and future automation roadmap.

## Critical Rules

1. Do not touch SaaS flows unless explicitly approved.
1. Do not touch operational logic unless required by the approved task.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not weaken tenant isolation.
1. Do not bypass permissions or platform owner checks.
1. Do not merge platform user, SaaS account, and tenant user concepts.
1. Do not change landing page implementation unless explicitly approved.
1. Do not add a KMS regenerate button until explicitly approved.
1. Do not push or commit unless explicitly requested.
1. Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or warning-level debug spam on normal requests.

## KMS Policy

Every implementation must include a Knowledge Impact Assessment:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If included docs change, regenerate:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

## Development Workflow

1. Read this file first.
2. Read `docs/TIS_MASTER_CONTEXT.md` and `docs/PROJECT_STATE.md`.
3. Read `docs/engineering/README.md`.
4. Read `docs/engineering/DEVELOPMENT_STANDARDS.md`.
5. Read `docs/engineering/AI_OPTIMIZATION_GUIDE.md`.
6. Read `docs/engineering/AI_CODING_WORKFLOW.md`.
7. Read relevant engineering docs, ADRs, module history, and supporting docs.
8. Inspect code before editing.
9. Keep changes scoped.
10. Update KMS docs when meaningful behavior, architecture, product state, module map, repository ownership, data model, design philosophy, roadmap, governance, decision traceability, automation, lifecycle, or workflow changes.
11. Regenerate PDF if included source docs changed.
12. Run validation.
13. Report KIA in final response.

## Landing Page Situation

The public landing implementation is in `tis-landing-website/`. Marketing docs live under `docs/marketing/`. Do not modify legacy FastAPI landing files unless explicitly approved.

## Next Planned Work

After Phase 2C review, a possible future enhancement is an explicit owner-only regenerate action. It should rebuild the PDF from reviewed Markdown source files only and must not silently rewrite Markdown source docs.

# docs/DOCUMENTATION_UPDATE_POLICY.md

# TIS Documentation Update Policy

This policy defines the non-negotiable rules for keeping the TIS Knowledge Management System reliable.

## Source Of Truth

Markdown files under `docs/` are the source of truth.

The PDF booklet at `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot. It must never be edited manually. It must be regenerated whenever included Markdown source files change.

## Required Knowledge Impact Assessment

Every approved implementation must end with a Knowledge Impact Assessment (KIA).

Required final report template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until the KIA is included, even when no documentation changes are needed.

## When Documentation Must Be Updated

Update documentation when a task changes:

1. product vision or positioning,
1. architecture or module boundaries,
1. SaaS signup, login, onboarding, billing, payment, or provisioning flows,
1. platform owner or permission behavior,
1. operational workflows such as calendar, planning, timetable, teachers, subjects, or observations,
1. deployment assumptions,
1. branch/release/project status,
1. landing page source-of-truth or visual strategy,
1. roadmap or known issues,
1. documentation system behavior.

## Which Docs To Update

Use this guide:

1. `docs/AI_PROJECT_CONTEXT.md`: compact onboarding context for future AI coding conversations; update when the high-level project situation changes.
1. `docs/TIS_MASTER_CONTEXT.md`: durable product, architecture, workflow, roadmap, and critical rules.

1. `docs/PROJECT_STATE.md`: current branch strategy, priorities, milestone status, known issues, and next work.
1. `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.
1. `docs/adr/`: major architectural or product decisions.
1. `docs/history/`: deeper module-specific before/after history.
1. Supporting docs under `docs/marketing/` or other folders when those areas change.

## ADR Rule

Create or update an ADR when a change affects a major long-term decision, including identity boundaries, payment architecture, provisioning strategy, deployment architecture, public website architecture, documentation governance, or visual system strategy.

Do not create ADRs for routine bug fixes unless they introduce or reverse a significant decision.

## Module History Rule

Update `docs/history/<module>/` when a meaningful module area changes and the previous documented state should be preserved.

`docs/CHANGE_HISTORY.md` remains the chronological summary. Module history stores deeper context.

## PDF Regeneration

Regenerate the PDF with:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

The generator must remain dependency-light:

1. use existing `reportlab`,
1. no LaTeX,
1. no Playwright or Chromium,
1. no external network calls,
1. no system font dependency.

## Reporting Rule

Every final implementation report must mention:

1. whether knowledge was impacted,
1. which docs changed,
1. whether change history changed,
1. whether an ADR was needed,
1. whether module history changed,
1. whether the PDF was regenerated,
1. whether `AI_PROJECT_CONTEXT.md` changed,
1. validation results.

## App Behavior Rule

The application may later detect documentation freshness and expose docs through owner-only protected routes. It must not silently rewrite Markdown source docs. Source docs are edited through reviewed development work.

# docs/TIS_MASTER_CONTEXT.md

# TIS Master Context

Documentation version: 3.0

Last major context update: 2026-06-27

## Product Identity

TIS stands for Teacher Information System. It is a developing SaaS academic operations platform for schools, school groups, academic leaders, supervisors, and platform owners who need one trusted place for academic staffing, teacher records, planning, calendars, observations, branch context, and future intelligence.

TIS is not only a teacher directory. It is intended to become the operational backbone for academic decision-making across a school or multi-branch organization.

Public product presence:

1. Public website: `https://tisplatform.com`
1. Application portal: `https://app.tisplatform.com`
1. Operational login: `/login`
1. SaaS signup: `/saas/signup`
1. SaaS login: `/saas/login`
1. SaaS account area: `/saas/account`

## Product Vision

TIS helps schools move away from scattered spreadsheets and disconnected operational files. The product vision is to give academic leaders a structured, secure, and tenant-isolated platform where teacher information, teaching loads, staffing needs, observations, calendars, and branch-level context can be managed together.

As the platform matures, TIS should also become the trusted data foundation for AI-assisted academic operations. Future AI features should depend on verified school data, clear permissions, and careful subscription packaging.

## Business Goal

The business goal is to establish TIS as a subscription-based SaaS platform for academic operations. The platform should support school onboarding, plan selection, payment, provisioning, account management, and scalable multi-tenant usage.

Near-term business priorities:

1. Convert interested schools from public landing page traffic into SaaS accounts.
1. Support clear onboarding from signup through school setup.
1. Provide reliable billing and payment status visibility.
1. Enable platform owners to manage pending organizations and provisioning.
1. Preserve trust by keeping tenant data isolated and operational flows stable.

## Educational Goal

The educational goal is to reduce administrative friction so schools can make better academic decisions. TIS should help leadership teams identify staffing gaps, workload problems, incomplete academic coverage, observation follow-up needs, and calendar conflicts earlier.

The platform should support educational quality by making academic operations easier to see, review, and improve.

## Target Customers

Primary customers:

- Private schools
- Multi-branch school groups
- Academic leadership teams
- Principals and vice principals
- Department heads and supervisors
- School operations and HR-adjacent academic staff

Internal platform users:

- Platform Owner
- Platform Co-Owner
- Platform Developer

Tenant users:

- Administrators
- Coordinators
- Supervisors
- Teachers and academic staff, depending on enabled workflows

## FastAPI App Architecture

The operational TIS application is a FastAPI application at the repository root. It uses Python, SQLAlchemy, Jinja templates, static assets, and modular routers.

Core app areas:

- `main.py`: primary FastAPI app, route registration, app-level workflows, startup checks, platform console routes, dashboard and configuration flows.
- `models.py`: main SQLAlchemy models.
- `database.py`: database connection/session setup.
- `db_migrations.py`: local schema migration and repair logic.
- `auth.py`: authentication, role normalization, platform identity helpers, permission helpers, session handling.
- `authorization.py`: route permission rules and access-denied response helpers.
- `permission_registry.py`: permission groups, labels, defaults, developer-assignable permissions, and system owner permissions.
- `role_permission_service.py`: role permission persistence and related helpers.

1. `ui_shell.py`: shared application shell context, navigation, visual identity, and page metadata.
1. `routers/`: modular feature routers for users, teachers, subjects, planning, timetable, academic calendar, and observations.
1. `templates/`: Jinja templates for the operational app and SaaS app pages.
1. `static/`: CSS, JavaScript, images, branding assets, and generated public artifacts.
1. `tests/`: pytest coverage for tenant isolation, SaaS phases, platform access, permissions, email, branding, and related workflows.

Important operational route families:

1. `/login`: operational app login.
1. `/dashboard`: tenant operational dashboard.
1. `/platform`: platform console for platform identities.
1. `/system-configuration`: branch, year, branding, role permissions, and configuration workflows.
1. `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, `/observations`: core academic operations.

## Next.js Landing Architecture

The public marketing website lives in `tis-landing-website/`. It is separate from the FastAPI operational portal.

Landing architecture:

1. Runtime: Next.js / Node.
1. Public website domain: `https://tisplatform.com`.
1. Local development URL: `http://localhost:3000`.
1. Main page: `tis-landing-website/src/app/page.tsx`.
1. App layout: `tis-landing-website/src/app/layout.tsx`.
1. Global styles: `tis-landing-website/src/app/globals.css`.
1. Logo component: `tis-landing-website/src/components/tis-logo.tsx`.
1. Public assets: `tis-landing-website/public/`.

The application portal remains separate:

1. App domain: `https://app.tisplatform.com`.
1. Runtime: FastAPI / Python with a relational database.

Legacy landing files in the FastAPI app are not the source of truth:

1. `templates/landing.html`
1. `static/landing/landing.css`

Those legacy files must not be modified unless explicitly approved.

## Multi-Tenant SaaS Strategy

TIS is designed as a multi-tenant SaaS platform. Tenant isolation is a critical rule. School groups, branches, users, academic years, teachers, planning data, timetable data, observations, and configuration records must remain scoped to the correct organization and branch context.

The platform distinguishes between:

1. Platform identities: platform owners, co-owners, and developers.
1. Tenant identities: users belonging to a school group and branch context.
1. SaaS account identities: accounts that move through signup, onboarding, billing, and provisioning.

Platform users may inspect or switch organization context through controlled platform workflows. Tenant users must remain inside their authorized school, branch, academic year, and permission scope.

Critical tenant strategy:

1. Do not weaken tenant isolation.
1. Do not bypass permission checks.
1. Do not assume a platform identity is a tenant identity.
1. Do not assume a SaaS account is already provisioned into an operational tenant.
1. Keep onboarding, billing, and tenant provisioning as distinct stages.

## SaaS Routes And Account Experience

Core SaaS routes:

1. `/saas/signup`: public SaaS account creation.
1. `/saas/login`: SaaS account login.
1. `/saas/account`: SaaS account dashboard.

Related SaaS areas include plan selection, onboarding organization details, contacts, branch setup, academic setup, onboarding review, billing status, checkout summary, checkout return, checkout cancel, sessions, security, and profile pages.

Platform owner SaaS administration exists under `/saas-admin` for pending organizations, payments, and provisioning workflows.

## M1-M5 Completed Milestone Summary

M1: Identity and SaaS foundation

1. Established core SaaS account concepts.
1. Added initial signup/login/account flows.
1. Clarified separation between platform, tenant, and SaaS account identities.
1. Added supporting tests for identity and SaaS phase behavior.

M2: Onboarding foundation

1. Added structured onboarding stages for organization information, contacts, branches, academic setup, and review.
1. Improved the path from SaaS signup toward a provisionable organization.
1. Preserved the distinction between pending SaaS organizations and operational tenants.

M3: Billing and plan foundation

1. Added pricing, plan catalog, billing status, checkout summary, return, and cancel flows.
1. Created billing/payment service modules to keep payment logic separate from operational academic logic.
1. Prepared the platform for subscription-based SaaS packaging.

M4: Provisioning foundation

1. Added pending organization and provisioning workflows for platform owners.
1. Added provisioning queue concepts and retry/run operations.
1. Created a controlled path for turning a pending SaaS organization into an operational school context.

M5: Platform access, permissions, and owner controls

1. Strengthened platform identity handling.
1. Added platform owner/developer concepts and owner management controls.
1. Improved permission boundaries for platform and tenant users.
1. Added tests around platform access and role permissions.

## Paddle And Payment Architecture Summary

The payment architecture is organized under the `saas/` package.

Key modules:

1. `saas/pricing_service.py`: plan/pricing behavior.
1. `saas/payment_service.py`: payment-related service logic.
1. `saas/paddle_client.py`: Paddle integration boundary.
1. `saas/billing_service.py`: billing status and related account billing behavior.
1. `saas/currency_service.py`: currency-related helpers.
1. `saas/router.py`: SaaS and SaaS admin routes.

Architecture rules:

1. Keep Paddle-specific details behind service/client boundaries.
1. Do not mix payment logic into academic operations.
1. Treat payment status, onboarding status, and provisioning status as related but separate concepts.
1. Use platform owner admin views for payment/provisioning oversight.
1. Avoid changing live SaaS payment flows unless the task explicitly requires it.

## Tenant Provisioning Summary

Tenant provisioning turns a pending SaaS organization into an operational TIS school context. Provisioning should create or connect the required organization records, branches, users, and initial tenant setup while preserving auditability and data isolation.

Key provisioning concepts:

1. Pending organization: SaaS onboarding entity not yet fully operational.

1. Provisioning job: controlled action to create or update operational tenant structures.
1. Platform owner review: human oversight before or during provisioning.
1. Retry behavior: failed provisioning work should be recoverable without corrupting tenant data.

Provisioning rules:

1. Do not directly create operational tenant data from public signup without the approved provisioning path.
1. Keep provisioning idempotent where possible.
1. Log or surface errors clearly.
1. Do not merge tenant data across school groups.

## Landing Page Strategy

The public landing page exists to explain the product, build trust, capture demand, and route interested schools into SaaS signup or demo request flows.

Source of truth:

1. The public website implementation is `tis-landing-website/`.
1. Marketing content references live in `docs/marketing/`.

Landing priorities:

1. Explain the problem of scattered academic operations.
1. Present TIS as a connected academic operations platform.
1. Show credible platform capabilities.
1. Support early access, demo requests, and signup pathways.
1. Keep the landing page separate from operational app templates.

Landing rule:

1. Do not change landing page design, copy, or architecture during operational backend tasks unless explicitly approved.

## Customer Experience Roadmap

Near-term customer experience:

1. Clear signup and login path.
1. Smooth onboarding for organization, contacts, branches, and academic setup.
1. Transparent billing/plan status.
1. Clear provisioning status after checkout or owner review.
1. Reliable operational login once provisioned.

Medium-term customer experience:

1. Better account self-service.
1. Clearer subscription lifecycle.
1. More guided onboarding and implementation support.

1. Improved platform owner visibility into pending organizations and payment status.

Long-term customer experience:

1. AI-assisted academic planning and decision support.
1. Subscription-gated advanced analytics.
1. Intelligent recommendations based on verified tenant data.
1. Richer executive visibility across branches and academic years.

## Knowledge Management System

TIS uses a Knowledge Management System (KMS) to preserve source-of-truth documentation, current project state, decision history, module history, and generated snapshots.

KMS source documents:

1. `docs/AI_PROJECT_CONTEXT.md`: first-read compact onboarding context for future Codex and ChatGPT conversations.
1. `docs/TIS_MASTER_CONTEXT.md`: durable product, architecture, workflow, roadmap, and critical rules.
1. `docs/PROJECT_STATE.md`: living project state.
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`: mandatory KMS and Knowledge Impact Assessment rules.
1. `docs/CHANGE_HISTORY.md`: chronological summary of meaningful changes.
1. `docs/adr/`: Architecture Decision Records.
1. `docs/history/`: module-specific history.
1. `docs/engineering/`: engineering handbook with module map, repository architecture, user/system flows, and developer onboarding.

PDF philosophy:

1. Markdown files are the source of truth.
1. The PDF booklet is a generated snapshot.
1. The PDF must never be edited manually.
1. The PDF must be regenerated when included Markdown source docs change.
1. The PDF should later be served through owner-only protected routes.

Generated KMS artifacts:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`
1. `static/docs/docs_manifest.json`

Owner-only app access:

1. `/platform/knowledge`: read-only Platform Owner Knowledge Center.
1. `/platform/knowledge/booklet`: protected inline PDF view.
1. `/platform/knowledge/booklet/download`: protected PDF download.

The Knowledge Center is protected by the existing Platform Owner access pattern. It is not public, not a landing page, and does not regenerate or rewrite source docs.

Engineering handbook:

1. `docs/engineering/TIS_MODULE_MAP.md` maps product/system modules and guardrails.
1. `docs/engineering/REPOSITORY_ARCHITECTURE.md` explains repository ownership and risky files.
1. `docs/engineering/USER_AND_SYSTEM_FLOWS.md` documents end-to-end customer, SaaS, payment, provisioning, operational, platform owner, KMS, and developer flows.
1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md` explains data areas, ownership boundaries, and tenant isolation rules.
1. `docs/engineering/DEVELOPMENT_STANDARDS.md` defines non-negotiable engineering rules.
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md` defines design direction for operational, platform, SaaS, Knowledge Center, and landing surfaces.
1. `docs/engineering/PRODUCT_ROADMAP.md` records completed, current, next, and future roadmap.
1. `docs/engineering/REJECTED_DECISIONS.md` records significant rejected alternatives.
1. `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md` defines future visual documentation standards.
1. `docs/engineering/AI_OPTIMIZATION_GUIDE.md` guides future AI assistants.
1. `docs/engineering/PROJECT_GOVERNANCE.md` defines ownership, approvals, quality gates, documentation gates, and traceability.
1. `docs/engineering/KNOWLEDGE_LIFECYCLE.md` defines documentation, engineering, approval, review, release, and maintenance lifecycle.
1. `docs/engineering/DOCUMENTATION_AUTOMATION.md` defines current automation and future automation rules.
1. `docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md` formalizes KIA.
1. `docs/engineering/SELF_EVOLVING_WORKFLOW.md` defines the official task-to-deployment workflow.
1. `docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md` explains KMS propagation.
1. `docs/engineering/AI_CODING_WORKFLOW.md` defines future AI coding discipline.
1. `docs/engineering/FUTURE_AUTOMATION_ROADMAP.md` records future automation opportunities.

KMS v1.0 status:

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation. Future work should improve automation, search, visuals, route inventories, test strategy docs, deployment runbooks, and deeper module guides only through approved phases.

## Development Workflow

Default workflow for approved implementation tasks:

1. Inspect the relevant code and docs before editing.
2. Keep changes scoped to the approved task.
3. Preserve tenant isolation, permission checks, SaaS flows, and landing page boundaries.
4. Update tests or add focused tests when behavior changes.
5. Complete the Knowledge Impact Assessment.
6. Update relevant Markdown docs.

7. Update change history, ADRs, module history, and AI project context when needed.
8. Regenerate the documentation PDF if included docs changed.
9. Run reasonable validation.
10. Report code changes, docs changes, KIA, validation, assumptions, and known issues.

## Knowledge Impact Assessment Rule

Every approved implementation must:

1. Assess knowledge impact.
2. Update relevant Markdown docs.
3. Update `docs/CHANGE_HISTORY.md` for meaningful changes.
4. Create or update ADRs when major decisions change.
5. Update module history when a module's documented state changes.
6. Update `docs/AI_PROJECT_CONTEXT.md` when high-level AI onboarding context changes.
7. Regenerate the PDF booklet if included docs changed.
8. Mention KIA details in the final report.

A task is not complete until the KIA is assessed.

Required final report KIA template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

The generated booklet output is:

1. `static/docs/TIS_Project_Reference_Booklet.pdf`

## Critical Rules Codex Must Follow

1. Do not touch SaaS flows unless the task explicitly requires it.
1. Do not touch operational logic unless the approved task requires it.
1. Do not touch database migrations or `tis.db` unless explicitly approved.
1. Do not weaken tenant isolation.
1. Do not bypass route permissions or platform owner checks.
1. Do not merge platform user, SaaS account, and tenant user concepts.
1. Do not change the landing page design or legacy landing files unless explicitly approved.
1. Do not add KMS regenerate behavior unless explicitly approved.
1. Do not expose the KMS PDF through direct public static links in the app UI.
1. Do not push or commit unless explicitly requested.

1. Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or normal-request diagnostic logging that can trigger Render restarts and 502s.
1. Prefer conservative, dependency-light automation.
1. Use `reportlab` for the documentation PDF generator.
1. Do not require LaTeX, Playwright, Chromium, external network calls, or system font dependencies for PDF generation.
1. Always include a Knowledge Impact Assessment in implementation final reports.

# docs/PROJECT_STATE.md

# TIS Project State

## Last Updated

Last updated: 2026-06-27

Update this file after every meaningful milestone, active development change, roadmap shift, known issue change, or documentation/KMS change.

## Current Branch Strategy

Current working branch assumption: `dev`.

Branch strategy:

1. Development work should happen on `dev` unless the owner explicitly requests another branch.
1. Production/live branch is assumed to be separate from active development.
1. Confirm production branch before any deployment, merge, or production-sensitive change.
1. Do not push, merge, or commit unless explicitly requested.
1. Preserve unrelated local changes.

## Production / Live Branch Assumption

The live production branch is assumed to be the branch deployed to the public app environment, while `dev` is the active development branch. This assumption must be confirmed before deployment.

Production domains:

1. Public website: `https://tisplatform.com`
1. Application portal: `https://app.tisplatform.com`

## Completed Milestones

M1: Identity and SaaS foundation

1. Core SaaS signup/login/account concepts established.
1. Platform, tenant, and SaaS account identities separated.
1. Identity and SaaS phase tests present.

M2: Onboarding foundation

1. SaaS onboarding flow covers organization, contacts, branches, academic setup, and review.
1. Pending organization concept supports pre-provisioning state.

M3: Billing and plan foundation

1. Plan catalog, checkout, billing status, checkout return, and checkout cancel flows exist.
1. Payment and billing code is isolated under `saas/` service modules.

M4: Provisioning foundation

1. Platform owner provisioning views and actions exist.
1. Pending organization review and provisioning queue behavior exist.
1. Provisioning retry/run operations are present.

M5: Platform access and owner controls

1. Platform owner and platform developer identities exist.
1. Platform console and owner/developer management controls exist.
1. Permission registry and platform access tests support this boundary.

Documentation/KMS milestones:

1. Phase 1 documentation foundation completed and pushed to `dev`.
1. Phase 2A and Phase 2B KMS foundation approved for implementation.
1. Phase 2C Platform Owner Knowledge Center completed and pushed to `dev`.
1. KMS v3.0 Phase 3A Engineering Handbook approved for implementation.
1. KMS v3.0 Phase 3B approved for implementation.
1. KMS v3.0 Phase 3C approved for implementation.
1. KMS v3.0 Phase 3D final phase approved for implementation.

## Current Priority

Current priority: upgrade the generated reference booklet into a true TIS Engineering Handbook.

Phase 2A and Phase 2B scope:

1. Create documentation update policy.
1. Create change history.
1. Create ADR foundation and initial accepted ADRs.
1. Create module history foundation.
1. Create AI project context.
1. Update master context, project state, and documentation index.
1. Update PDF generator to include KMS docs and manifest metadata.
1. Regenerate `static/docs/TIS_Project_Reference_Booklet.pdf`.

Phase 2C completed scope:

1. Added read-only `knowledge_service.py` as the single KMS app access layer.
1. Added owner-protected `/platform/knowledge` page.
1. Added owner-protected PDF view/download routes.
1. Added an owner-only Platform Console card.
1. Added platform knowledge module history.
1. Regenerated the PDF and manifest after documentation updates.

Still out of scope:

- Regenerate button.
- Additional app routes beyond the approved read-only Knowledge Center routes.
- `ui_shell.py` and `authorization.py` changes unless separately approved.
- SaaS flows.
- Operational logic.
- Database, migrations, or `tis.db`.
- Landing page implementation.

KMS v3.0 Phase 3A scope:

- Add complete TIS module map.
- Add repository architecture map.
- Add end-to-end user/system workflows.
- Add clear AI/human developer onboarding structure.
- Update generator to include engineering docs.
- Regenerate the PDF and manifest.

KMS v3.0 Phase 3B scope:

- Add database architecture overview.
- Add development standards and non-negotiable rules.
- Add UI/UX and design philosophy.
- Add product roadmap.
- Strengthen AI/human developer onboarding guidance.
- Update generator to include the new engineering docs.
- Regenerate the PDF and manifest.

KMS v3.0 Phase 3C scope:

- Add rejected architectural decisions.
- Add visual documentation framework.
- Add AI optimization guide.
- Add project governance and decision traceability.
- Update generator to include the new engineering docs.
- Regenerate the PDF and manifest.

KMS v3.0 Phase 3D final scope:

- Add knowledge lifecycle documentation.
- Add documentation automation guide.
- Add formal KIA standard.
- Add self-evolving workflow.

1. Add documentation dependency map.
1. Add AI coding workflow.
1. Add future automation roadmap.
1. Regenerate the PDF and manifest.

## Current Known Issues

Known issues and watch points:

1. KMS policy depends on future developers and AI agents consistently completing the Knowledge Impact Assessment.
1. Generated PDF can become stale if included Markdown docs change without regeneration.
1. The owner-only Knowledge Center is implemented as read-only; there is no regenerate button yet.
1. Public static storage is not sufficient access control for sensitive docs; Phase 2C should serve docs through protected owner-only routes.
1. Render deployment constraints should continue to guide dependency choices.
1. Production memory must be treated as a hard constraint. The 2026-06-27 Render restart/502 investigation found two avoidable memory risks: observation diagnostics doing extra production template renders and global location lookup parsing a 47 MB dataset into a complete in-memory index for simple picker requests. Local stabilization changes now gate observation diagnostics and use scoped location loading; future work must follow the Production Memory and Render Stability standards.
1. Broad filesystem scans may warn about `tis_scope_test_5i3yf0h5/` access denial.

## Next Planned Work

Next planned work after Phase 2C review:

1. Review final KMS v1.0 readiness, handbook completeness, AI readiness, PDF readability, and manifest inclusion.
1. Approve corrections if needed.
1. Later consider an explicit owner-only regenerate workflow.
1. Review, commit, and deploy the production memory stabilization changes when approved, then monitor Render memory, restart count, and route-level 502s after deployment.

## Landing Page Baseline Situation

The public landing page source of truth is:

1. `tis-landing-website/`

Marketing docs:

1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/marketing/tis_landing_page_master_content.md`

Relevant ADRs:

1. `docs/adr/0001-separate-nextjs-landing-website.md`
1. `docs/adr/0007-landing-page-visual-system-strategy.md`

Legacy FastAPI landing files are not the current public website source of truth:

- `templates/landing.html`
- `static/landing/landing.css`

Do not modify landing page design, landing copy, or legacy landing files unless explicitly approved.

## Knowledge Update Policy

Every approved implementation must complete the KIA:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included docs change, regenerate:

- `static/docs/TIS_Project_Reference_Booklet.pdf`

## Scope Guardrails

- Do not touch SaaS flows unless explicitly approved.
- Do not touch operational logic unless required by the approved task.
- Do not touch database migrations or `tis.db` unless explicitly approved.
- Do not change the landing page unless explicitly approved.
- Do not add Platform Owner Knowledge Center routes until reviewed and approved.
- Do not add a KMS regenerate button until separately approved.
- Do not implement Phase 3B until reviewed and approved.
- Do not implement Phase 3C until reviewed and approved.
- Do not implement Phase 3D until reviewed and approved.
- Do not begin further KMS work until reviewed and approved.
- Do not commit or push unless explicitly requested.

# docs/CHANGE_HISTORY.md

# TIS Change History

This file is the chronological summary of meaningful TIS changes. It does not replace module history under `docs/history/`; it gives reviewers, developers, Codex, and ChatGPT a fast timeline of what changed and why.

Newest entries should be added first.

## Entry Template

```
## YYYY-MM-DD - Short Change Title

Area/module:
Previous state:
New state:
Reason:
Files changed:
Documentation updated:
PDF regenerated:
AI project context updated:
Reviewer/approval notes:
```

## 2026-06-27 - Added Production Memory Stability Guardrails

Area/module: Operational app stability, observations, global location lookup, Render deployment constraints, and engineering standards

Previous state: Production traffic could hit avoidable memory pressure. The observations page included diagnostic stage logging and extra template rendering in the normal request path, and the global location picker could parse a 47 MB reference dataset into a complete in-memory index for simple picker requests. KMS standards did not yet explicitly forbid unbounded caches, duplicate production template renders, or normal-request diagnostic warning spam.

New state: The local stabilization patch gates observation diagnostics behind `TIS_OBSERVATION_DIAGNOSTICS`, removes duplicate observation template pre-renders, and changes location lookup behavior to use streaming/scoped country loading for normal country, region, city, and validation requests. KMS now documents strict production memory and Render stability rules.

Reason: Render logs showed app restarts and user-facing 502s around normal app navigation after deployment. A 512 MB service can be enough for TIS only if the app avoids unnecessary full-dataset memory loads, duplicate rendering, and production debug noise.

Files changed:

1 `location_service.py`

1 `routers/observations.py`

1 `docs/engineering/DEVELOPMENT_STANDARDS.md`

1 `docs/PROJECT_STATE.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/CHANGE_HISTORY.md`

1 `docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md`

1 `static/docs/TIS_Project_Reference_Booklet.pdf`

```
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Local changes only. No commit, push, migration, database change, SaaS route change, billing change, tenant logic change, or deployment was performed.

## 2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3C documented rejected decisions, visual documentation framework, AI optimization, governance, and traceability. It still needed a complete self-evolving lifecycle standard that ties implementation, validation, KIA, documentation updates, generated artifacts, review, commit, push, and deployment together.

New state: TIS now has final Phase 3D docs for knowledge lifecycle, documentation automation, KIA standard, self-evolving workflow, documentation dependency map, AI coding workflow, and future automation roadmap. This completes the KMS v1.0 lifecycle foundation.

Reason: Ensure every future approved implementation naturally keeps KMS synchronized without relying on uncontrolled app-side rewriting of source docs.

Files changed:

```
docs/engineering/KNOWLEDGE_LIFECYCLE.md
docs/engineering/DOCUMENTATION_AUTOMATION.md
docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
docs/engineering/SELF_EVOLVING_WORKFLOW.md
docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md
docs/engineering/AI_CODING_WORKFLOW.md
docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
docs/engineering/README.md
docs/TIS_MASTER_CONTEXT.md
docs/AI_PROJECT_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
docs/CHANGE_HISTORY.md
docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3D final phase only. SaaS flows, landing page code, database models, migrations, `tis.db`, routes, commits, and pushes remain out of scope.

## 2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3B documented database architecture, development standards, UI/UX design philosophy, roadmap, and stronger onboarding guidance. It did not yet preserve rejected decisions, visual documentation standards, a definitive AI optimization guide, project governance, or explicit decision traceability.

New state: TIS now has Phase 3C engineering docs for rejected decisions, visual documentation framework, AI optimization, project governance, and decision traceability. The PDF generator includes these docs.

Reason: Future developers and AI assistants need to understand why TIS became what it is, not only what currently exists.

Files changed:

1 `docs/engineering/REJECTED_DECISIONS.md`

1 `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md`

1 `docs/engineering/AI_OPTIMIZATION_GUIDE.md`

1 `docs/engineering/PROJECT_GOVERNANCE.md`

1 `docs/engineering/README.md`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/PROJECT_STATE.md`

1 `docs/README.md`

1 `docs/CHANGE_HISTORY.md`

1 `docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md`

1 `scripts/generate_docs_pdf.py`

1 `static/docs/TIS_Project_Reference_Booklet.pdf`

1 `static/docs/docs_manifest.json`

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

## 2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3A added module map, repository architecture, user/system flows, and onboarding structure. The handbook still needed database architecture, development standards, UI/UX philosophy, roadmap, and

stronger human/AI guidance.

New state: TIS now has Phase 3B engineering docs for database architecture, development standards, UI/UX design philosophy, and product roadmap. Core KMS docs and AI onboarding guidance reference these layers, and the PDF generator includes them.

Reason: Make the generated booklet more useful for new senior developers, Codex conversations, ChatGPT conversations, and future technical reviewers.

Files changed:

```
docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
docs/engineering/DEVELOPMENT_STANDARDS.md
docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
docs/engineering/PRODUCT_ROADMAP.md
docs/engineering/README.md
docs/TIS_MASTER_CONTEXT.md
docs/AI_PROJECT_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
docs/CHANGE_HISTORY.md
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

## 2026-06-26 - Added KMS v3.0 Engineering Handbook

Area/module: Knowledge Management System and engineering onboarding

Previous state: The generated booklet included KMS source documents, ADRs, module history, AI context, and the Knowledge Center foundation, but it did not fully onboard a new human developer or future Codex/ChatGPT conversation into TIS modules, repository architecture, and end-to-end flows.

New state: TIS now has an engineering handbook layer with a complete module map, repository architecture guide, user/system flow guide, and engineering onboarding index. The PDF generator includes these docs and emits documentation version 3.0.

Reason: Make the generated booklet a true TIS Engineering Handbook rather than only a documentation bundle.

Files changed:

```
docs/engineering/README.md
docs/engineering/TIS_MODULE_MAP.md
```

```
docs/engineering/REPOSITORY_ARCHITECTURE.md
docs/engineering/USER_AND_SYSTEM_FLOWS.md
docs/TIS_MASTER_CONTEXT.md
docs/AI_PROJECT_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
docs/CHANGE_HISTORY.md
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

## 2026-06-26 - Added Platform Owner Knowledge Center

Area/module: Platform Knowledge Center and KMS access

Previous state: TIS had KMS source docs, ADRs, module history, a generated PDF booklet, and a manifest, but no protected in-app owner page for KMS status or booklet access.

New state: TIS now has a read-only Platform Owner Knowledge Center with KMS health score, manifest metadata, freshness detection, source document status, coverage checks, latest change-history entries, ADR list, module history areas, KIA checklist, and protected PDF view/download routes.

Reason: Platform owners need an internal utility for verifying KMS health and accessing the generated PDF without exposing direct public static links.

Files changed:

```
knowledge_service.py
main.py
templates/platform_knowledge_center.html
templates/platform_console.html
scripts/generate_docs_pdf.py
docs/TIS_MASTER_CONTEXT.md
docs/PROJECT_STATE.md
docs/README.md
docs/CHANGE_HISTORY.md
docs/AI_PROJECT_CONTEXT.md
docs/history/platform-knowledge/README.md
```

- `docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md`
- `static/docs/TIS_Project_Reference_Booklet.pdf`
- `static/docs/docs_manifest.json`

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2C only. Regenerate button, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

## 2026-06-26 - Established Knowledge Management System Foundation

Area/module: Documentation and project knowledge management

Previous state: TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal change history, ADR system, module history foundation, KMS policy, manifest, or compact AI onboarding file.

New state: TIS now has a Knowledge Management System foundation with chronological change history, documentation update policy, ADR structure and initial accepted ADRs, module history folders, AI project context, updated source docs, and an expanded PDF generator.

Reason: Preserve project knowledge for future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and technical reviewers.

Files changed:

- `docs/CHANGE_HISTORY.md`
- `docs/DOCUMENTATION_UPDATE_POLICY.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/adr/README.md`
- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`
- `docs/adr/0003-paddle-payment-architecture.md`
- `docs/adr/0004-webhook-only-payment-confirmation.md`
- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- `docs/adr/0006-documentation-as-source-knowledge-management-system.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`
- `docs/history/README.md`
- `docs/history/*/README.md`
- `docs/history/provisioning/2026-06-26-kms-foundation.md`
- `docs/TIS_MASTER_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/README.md`
- `scripts/generate_docs_pdf.py`

```
static/docs/TIS_Project_Reference_Booklet.pdf
```

```
static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only. Platform Owner Knowledge Center, app routes, SaaS flows, database, migrations, landing page implementation, commits, and pushes remain out of scope.

# docs/adr/README.md

# Architecture Decision Records

ADRs record major TIS architectural and product decisions. They explain why the system is shaped the way it is.

## ADR Rules

1. Create an ADR for major decisions affecting architecture, identity, SaaS, payment, provisioning, deployment, documentation governance, or long-term product boundaries.
1. Do not use ADRs for ordinary bug fixes.
1. If a decision is replaced, mark the older ADR as Superseded and link the new ADR.
1. ADRs complement `docs/TIS_MASTER_CONTEXT.md`; they do not replace it.

## Status Values

1. Proposed
1. Accepted
1. Superseded
1. Deprecated

## ADR Index

1. `0001-separate-nextjs-landing-website.md`
1. `0002-separate-saas-identity-and-operational-users.md`
1. `0003-paddle-payment-architecture.md`
1. `0004-webhook-only-payment-confirmation.md`
1. `0005-delayed-tenant-provisioning-after-verified-payment.md`
1. `0006-documentation-as-source-knowledge-management-system.md`
1. `0007-landing-page-visual-system-strategy.md`

# docs/engineering/README.md

# TIS Engineering Handbook

This folder turns the TIS KMS into a practical engineering handbook. It is intended for new human developers, future Codex conversations, future ChatGPT conversations, project owners, platform owners, and reviewers.

## Read First

Recommended onboarding order:

1. `docs/AI_PROJECT_CONTEXT.md`
2. `docs/TIS_MASTER_CONTEXT.md`
3. `docs/PROJECT_STATE.md`
4. `docs/engineering/TIS_MODULE_MAP.md`
5. `docs/engineering/REPOSITORY_ARCHITECTURE.md`
6. `docs/engineering/USER_AND_SYSTEM_FLOWS.md`
7. Relevant ADRs under `docs/adr/`
8. Relevant module history under `docs/history/`

## Engineering Documents

1. `TIS_MODULE_MAP.md`: product and system module map with purpose, files, maturity, docs, risks, and guardrails.
1. `REPOSITORY_ARCHITECTURE.md`: repository structure and responsibilities.
1. `USER_AND_SYSTEM_FLOWS.md`: end-to-end public, SaaS, payment, provisioning, operational, platform owner, and KMS flows.
1. `DATABASE_ARCHITECTURE_OVERVIEW.md`: high-level data model relationships and isolation boundaries.
1. `DEVELOPMENT_STANDARDS.md`: non-negotiable engineering rules for humans and AI assistants.
1. `UI_UX_DESIGN_PHILOSOPHY.md`: design principles for operational app, platform owner tools, SaaS onboarding, Knowledge Center, and landing website.
1. `PRODUCT_ROADMAP.md`: completed, current, next, and future roadmap.
1. `REJECTED_DECISIONS.md`: significant rejected architectural/product alternatives and why they were declined.
1. `VISUAL_DOCUMENTATION_GUIDE.md`: future screenshot and diagram standards.
1. `AI_OPTIMIZATION_GUIDE.md`: definitive onboarding and operating guide for future AI assistants.
1. `PROJECT_GOVERNANCE.md`: ownership, approvals, quality gates, documentation gates, and decision traceability.
1. `KNOWLEDGE_LIFECYCLE.md`: documentation, engineering, approval, review, release, and maintenance lifecycle.
1. `DOCUMENTATION_AUTOMATION.md`: current and future documentation automation rules.
1. `KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md`: formal KIA standard with examples, decision tree, and failure cases.
1. `SELF_EVOLVING_WORKFLOW.md`: official task-to-deployment workflow.
1. `DOCUMENTATION_DEPENDENCY_MAP.md`: relationships and propagation rules across KMS documents.

1. `AI_CODING_WORKFLOW.md`: required workflow for future AI coding assistants.
1. `FUTURE_AUTOMATION_ROADMAP.md`: possible future automation improvements.

## Before Coding

Before changing code:

1. confirm the approved scope,
1. inspect affected files and tests,
1. read relevant ADRs,
1. read relevant module history,
1. preserve tenant isolation,
1. preserve identity boundaries,
1. avoid touching SaaS, landing, database, or migrations unless explicitly approved.

## After Coding

Every implementation must complete the Knowledge Impact Assessment:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If included source docs changed, regenerate:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

# docs/engineering/TIS_MODULE_MAP.md

# TIS Module Map

This map describes the known TIS modules, where they live, their maturity, related docs/ADRs, and the guardrails developers must respect.

## Platform Owner

Purpose: Own global TIS platform operations, owner/co-owner controls, platform developer accounts, cross-organization oversight, and protected KMS access.

Main files/folders:

- `auth.py`
- `main.py`
- `templates/platform_console.html`
- `templates/platform_knowledge_center.html`
- `knowledge_service.py`
- `permission_registry.py`

Maturity/status: Implemented for owner identity, platform console, developer/co-owner controls, and read-only Knowledge Center.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`
- `docs/history/platform-knowledge/`

Risks/guardrails:

- Do not treat platform developers as owners.
- Do not expose owner utilities to tenant users.
- Reuse `auth.is_platform_owner(...)` and existing owner access helpers.

## Platform Console

Purpose: Allow platform identities to inspect organizations, switch context, and manage platform owner/developer controls.

Main files/folders:

- `main.py`
- `templates/platform_console.html`
- `ui_shell.py`

Maturity/status: Implemented and active.

Related docs/ADRs:

- `docs/PROJECT_STATE.md`

Risks/guardrails:

- Context switching must not weaken tenant isolation.
- Owner-only management controls must remain owner-only.

## Knowledge Center

Purpose: Show KMS health, source coverage, PDF freshness, ADRs, module history, change history, and KIA policy to platform owners.

Main files/folders:

- `knowledge_service.py`
- `main.py`
- `templates/platform_knowledge_center.html`
- `static/docs/docs_manifest.json`
- `static/docs/TIS_Project_Reference_Booklet.pdf`

Maturity/status: Read-only Phase 2C implementation complete. No regenerate button.

Related docs/ADRs:

- `docs/adr/0006-documentation-as-source-knowledge-management-system.md`
- `docs/history/platform-knowledge/`

Risks/guardrails:

- Do not link directly to `/static/docs/...` from UI.
- Do not let the app rewrite Markdown source docs.
- Do not add regenerate behavior without approval.

## SaaS Account System

Purpose: Handle public SaaS signup/login/account profile, sessions, security, billing visibility, and onboarding status.

Main files/folders:

- `saas/router.py`
- `saas/service.py`
- `saas/models.py`
- `templates/saas/`

Maturity/status: M1-M5 foundation implemented.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`
- `docs/history/saas-onboarding/`

Risks/guardrails:

- Do not merge SaaS account identity with operational tenant users.

- Do not bypass verification/security/session boundaries.

## SaaS Onboarding

Purpose: Collect organization, contact, branch, academic setup, and review data before provisioning.

Main files/folders:

- `saas/router.py`
- `saas/service.py`
- `templates/saas/onboarding_*.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/history/saas-onboarding/`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Pending onboarding data is not the same as operational tenant data.
- Do not create live tenant records directly from public forms.

## Pending Organizations

Purpose: Represent SaaS organizations waiting for review, payment readiness, or provisioning.

Main files/folders:

- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `templates/saas/admin_pending_organizations.html`
- `templates/saas/admin_pending_organization_detail.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

Risks/guardrails:

- Keep pending state separate from provisioned tenant state.
- Preserve platform owner review and auditability.

## Plans And Pricing

Purpose: Define SaaS plan catalog and pricing behavior.

Main files/folders:

- `saas/pricing_service.py`

- `saas/router.py`
- `templates/saas/plan_catalog.html`
- `templates/saas/plan_selection.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/history/subscriptions/`
- `docs/adr/0003-paddle-payment-architecture.md`

Risks/guardrails:

- Plan changes must update docs and change history.
- Do not hard-code provider behavior outside service boundaries.

## Paddle / Payment

Purpose: Integrate subscription checkout, payment state, billing status, and provider events.

Main files/folders:

- `saas/paddle_client.py`
- `saas/payment_service.py`
- `saas/billing_service.py`
- `saas/currency_service.py`
- `saas/router.py`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0003-paddle-payment-architecture.md`
- `docs/adr/0004-webhook-only-payment-confirmation.md`
- `docs/history/subscriptions/`

Risks/guardrails:

- Webhook-confirmed payment state is authoritative.
- Checkout return alone must not trigger verified payment/provisioning.

## Provisioning

Purpose: Convert verified/approved pending organizations into operational tenant structures.

Main files/folders:

- `saas/provisioning_service.py`
- `saas/router.py`
- `templates/saas/admin_provisioning.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- `docs/history/provisioning/`

Risks/guardrails:

- Keep provisioning recoverable and reviewable.
- Do not merge tenants or create records in the wrong school group.

## Operational Login

Purpose: Authenticate operational users and route them into platform or tenant context.

Main files/folders:

- `main.py`
- `auth.py`
- `templates/login.html` if present through root templates

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Platform users and tenant users follow different context behavior.
- Preserve permission and active-user checks.

## Dashboard

Purpose: Give tenant users operational visibility into staffing, planning, reports, and current school context.

Main files/folders:

- `main.py`
- `templates/dashboard.html`
- related report/export helpers

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Dashboard data must remain scoped to tenant/branch/year context.

## Organizations / School Groups

Purpose: Represent tenant organizations and top-level school group context.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_schools.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Tenant isolation starts at school group boundaries.

## Branches

Purpose: Represent campuses/branches inside a school group.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_branches.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Branch scope affects teachers, users, planning, timetable, calendar, branding, and reports.

## Academic Years

Purpose: Scope operational data by academic year and active-year context.

Main files/folders:

- `models.py`
- `main.py`
- `templates/system_configuration_years.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Do not mix data across academic years.

## Users And Roles

Purpose: Manage tenant users, platform users, roles, profile data, active status, and branch/year context.

Main files/folders:

- `auth.py`
- `routers/users.py`
- `models.py`
- `templates/users.html`
- `templates/edit_user.html`

Maturity/status: Implemented with platform identity refinements.

Related docs/ADRs:

- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- Do not assign owner controls to developers or tenant users.
- Preserve role normalization.

## Permissions

Purpose: Control route/module actions by permission key and role package.

Main files/folders:

- `authorization.py`
- `permission_registry.py`
- `role_permission_service.py`
- `auth.py`
- `templates/system_configuration_role_permissions.html`

Maturity/status: Implemented and critical.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Do not bypass `authorization.enforce_route_permission`.
- Do not rely only on UI hiding for protected actions.

## Teachers

Purpose: Manage teacher records, qualifications, capacity, workloads, and profile-related academic staffing data.

Main files/folders:

```
routers/teachers.py
teacher_qualifications.py
teacher_capacity.py
models.py
templates/teachers.html
templates/edit_teacher.html
```

Maturity/status: Implemented and central to operational value.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Teacher data must remain tenant/branch/year scoped.
```

## Subjects

Purpose: Manage subject catalog, colors, qualifications, and planning/timetable relationships.

Main files/folders:

```
routers/subjects.py
subject_colors.py
models.py
templates/subjects.html
templates/edit_subject.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Subject changes can affect planning, timetable, teacher qualifications, and reports.
```

## Sections

Purpose: Represent class/section structures used by planning and timetabling.

Main files/folders:

```
routers/planning.py
models.py
templates/planning.html
```

Maturity/status: Implemented through planning workflows.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Section changes can affect timetable allocation and workload reporting.
```

## Workforce Planning

Purpose: Plan assignments, homeroom ownership, workloads, subject coverage, and staffing needs.

Main files/folders:

```
routers/planning.py
teacher_capacity.py
homeroom_defaults.py
templates/planning.html
```

Maturity/status: Implemented and evolving.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Planning logic must preserve branch/year scope and avoid accidental cross-year copies.
```

## Timetable

Purpose: Place planned lessons into weekly timetable grids and exports.

Main files/folders:

```
routers/timetable.py
timetable_logic.py
templates/timetable.html
templates/system_configuration_timetable.html
```

Maturity/status: Implemented and evolving.

Related docs/ADRs:

```
docs/history/workforce-planning/
```

Risks/guardrails:

```
Timetable rules interact with planning, sections, subjects, and teacher capacity.
```

## Academic Calendar

Purpose: Manage academic events, responsibilities, dates, exports, and branch/year scoped calendar views.

Main files/folders:

```
routers/academic_calendar.py
```

- `templates/academic_calendar.html`
- `templates/system_configuration_calendar.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/history/academic-calendar/`

Risks/guardrails:

- Calendar permissions and branch/year scoping must remain intact.

## Observations

Purpose: Support teacher observations, feedback, evidence, scoring, history, and supervision records.

Main files/folders:

- `routers/observations.py`
- `templates/observations.html`
- `templates/observation_form.html`
- `templates/observation_detail.html`
- `templates/observation_history.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- Observation records are sensitive and must remain tenant scoped.

## Reports / Dashboards

Purpose: Expose operational summaries, exports, allocation reports, and decision visibility.

Main files/folders:

- `main.py`
- report/export helper code
- `templates/dashboard.html`

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- `docs/history/workforce-planning/`

Risks/guardrails:

- Report data must be permission-checked and tenant scoped.

## Branding / Design Settings

Purpose: Manage organization logos, design settings, visual shell behavior, and platform visual controls.

Main files/folders:

- `branding_storage.py`
- `design_tokens.py`
- `visual_design.py`
- `ui_shell.py`
- `static/css/branding.css`
- `static/css/design-studio.css`
- `templates/system_configuration_logos.html`
- `templates/system_configuration_design.html`

Maturity/status: Implemented.

Related docs/ADRs:

- `docs/adr/0007-landing-page-visual-system-strategy.md`

Risks/guardrails:

- Do not confuse operational app branding with public landing website design.
- Protect uploaded/owned assets.

## Landing Website

Purpose: Public marketing and conversion surface for TIS.

Main files/folders:

- `tis-landing-website/`
- `docs/marketing/`

Maturity/status: Implemented as separate Next.js app.

Related docs/ADRs:

- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`
- `docs/marketing/landing_page_source_of_truth.md`

Risks/guardrails:

- Do not modify landing code during operational/backend tasks unless explicitly approved.
- Legacy FastAPI landing files are not the public source of truth.

## AI Future Roadmap

Purpose: Future subscription-gated AI-assisted planning, analytics, assessment generation, recommendations, and decision support.

Main files/folders:

- No dedicated AI production module yet.
- Future work should be documented before implementation.

Maturity/status: Roadmap / future.

Related docs/ADRs:

- `docs/TIS_MASTER_CONTEXT.md`
- future ADRs required before major AI architecture decisions.

Risks/guardrails:

- AI features must use verified tenant data and preserve privacy, permissions, and subscription boundaries.

# docs/engineering/REPOSITORY_ARCHITECTURE.md

# TIS Repository Architecture

This document explains the main repository areas, what each owns, and what must not be changed casually.

## Root FastAPI Application

### `main.py`

Responsibility: Primary FastAPI app, route registration, many app-level workflows, middleware, startup checks, platform console routes, dashboards, exports, system configuration, scope switching, and operational pages.

Do not change casually:

1. authentication/session flow,
1. platform owner access checks,
1. tenant scope handling,
1. route behavior shared by many modules,
1. startup/schema repair behavior.

### `auth.py`

Responsibility: Password hashing/verification, session cookies, current-user lookup, role normalization, platform identity helpers, permission helpers, email verification helpers, and tenant/platform identity boundaries.

Do not change casually:

1. `is_platform_owner`, `is_platform_user`, `is_platform_developer`,
1. role normalization,
1. session cookie behavior,
1. permission helper semantics.

### `authorization.py`

Responsibility: Protected route rules, permission enforcement middleware integration, access-denied responses.

Do not change casually:

1. route permission mapping,
1. public path patterns,
1. permission matching semantics.

### `database.py`

Responsibility: Database engine/session setup.

Do not change casually:

1. database URL handling,
1. session creation behavior,

1 engine configuration.

### models.py

Responsibility: Primary operational SQLAlchemy models for users, school groups, branches, teachers, planning, timetable, observations, configuration, and related app data.

Do not change casually:

1 tenant ownership fields,

1 platform user fields,

1 relationships used by existing workflows,

1 schema without matching migration/repair strategy.

### db_migrations.py

Responsibility: Local schema migration and repair logic used by the app.

Do not change casually:

1 migration ordering,

1 destructive schema operations,

1 production-sensitive repair behavior.

### permission_registry.py

Responsibility: Permission groups, labels, default role permissions, system owner permissions, developer-assignable permission boundaries.

Do not change casually:

1 owner/developer permissions,

1 defaults for managed roles,

1 permission keys referenced by routes/templates/tests.

### role_permission_service.py

Responsibility: Persistence and service helpers for role permission rows.

Do not change casually:

1 global vs school-scoped permission behavior,

1 role permission constraints.

### ui_shell.py

Responsibility: Shared application shell, navigation, page metadata, scoped organization/year context display, logos, visual design CSS, and permission-based nav generation.

Do not change casually:

1 navigation visibility,

1 platform-vs-tenant shell behavior,

- design-studio gating,
- scope display.

## Feature Routers: `routers/`

Responsibility: Modular operational route handlers:

- `routers/users.py`
- `routers/teachers.py`
- `routers/subjects.py`
- `routers/planning.py`
- `routers/timetable.py`
- `routers/academic_calendar.py`
- `routers/observations.py`

Do not change casually:

- tenant/branch/year scoping,
- permission checks,
- import/export behavior,
- bulk operations.

## SaaS Package: `saas/`

Responsibility: Public SaaS account flows, onboarding, plans, billing, Paddle integration, pending organizations, and provisioning.

Key files:

- `saas/router.py`
- `saas/service.py`
- `saas/models.py`
- `saas/pricing_service.py`
- `saas/payment_service.py`
- `saas/paddle_client.py`
- `saas/billing_service.py`
- `saas/provisioning_service.py`
- `saas/currency_service.py`
- `saas/oauth.py`

Do not change casually:

- identity separation,
- payment confirmation rules,
- provisioning readiness,

1. Paddle/webhook boundaries,
1. public signup/onboarding state transitions.

## Templates: `templates/`

Responsibility: Jinja templates for operational app, platform console, Knowledge Center, SaaS pages, system configuration, and feature workflows.

Do not change casually:

1. form action routes,
1. hidden scope fields,
1. permission-dependent controls,
1. app shell extension patterns.

## Static Assets: `static/`

Responsibility: Operational CSS/JS, images, branding assets, generated documentation PDF and manifest.

Important:

1. `static/docs/TIS_Project_Reference_Booklet.pdf` is a generated snapshot.
1. `static/docs/docs_manifest.json` is generated metadata.

Do not change casually:

1. generated docs manually,
1. shared CSS without checking all templates,
1. protected document access assumptions.

## Tests: `tests/`

Responsibility: Regression coverage for SaaS phases, tenant isolation, platform access, permissions, email, branding, and critical workflows.

Do not change casually:

1. tests should follow behavior, not hide regressions.
1. update tests when behavior intentionally changes.

## Docs: `docs/`

Responsibility: KMS source of truth, engineering handbook, ADRs, change history, module history, AI context, marketing docs.

Do not change casually:

1. historical records should preserve old/new state.
1. update docs through the KIA process.

## Scripts: `scripts/`

Responsibility: Maintenance scripts such as `scripts/generate_docs_pdf.py`.

Do not change casually:

1. PDF generator dependency assumptions,
1. source list behavior,
1. manifest metadata.

## Next.js Landing Website: `tis-landing-website/`

Responsibility: Public marketing website at `https://tisplatform.com`.

Do not change casually:

1. landing implementation during backend tasks,
1. visual system without approval,
1. public assets/copy without checking marketing docs and ADRs.

# docs/engineering/USER_AND_SYSTEM_FLOWS.md

# TIS User And System Flows

This document describes the major end-to-end flows a developer must understand before changing TIS.

## Public Customer Flow

Flow:

1. Public visitor opens `https://tisplatform.com`.
2. Visitor clicks a signup/get-started path.
3. Visitor reaches SaaS signup at `/saas/signup`.
4. SaaS account is created.
5. Email verification is completed when required.
6. User signs into SaaS account through `/saas/login`.
7. User enters `/saas/account`.
8. User completes organization onboarding:
    1. organization details,
    1. contacts,
    1. branches,
    1. academic setup,
    1. review.
    1. User selects a plan.
    2. User enters checkout.
    3. Paddle handles payment.
    4. Return/cancel page informs the user of checkout navigation result.
    5. Paddle webhook confirms payment.
    6. Local payment/billing state is updated.
    7. Pending organization becomes ready for provisioning.
    8. Platform owner reviews/runs provisioning.
    9. Operational tenant structures are created.
    10. Operational login becomes available through `/login`.

Guardrails:

1. Checkout return is not authoritative payment confirmation.
1. Public signup must not directly create operational tenant data.
1. Provisioning should occur only through the approved flow.

## SaaS Identity Flow

Flow:

1. User signs up through `/saas/signup`.
2. SaaS account/session records are created.
3. User signs in through `/saas/login`.
4. Account dashboard is available at `/saas/account`.
5. User can access profile, sessions, security, billing status, and onboarding state.

Important distinction:

1. SaaS account identity is not the same as operational tenant user identity.
1. Platform identity is also separate.

Guardrails:

1. Do not merge SaaS accounts with operational users unless an approved provisioning flow creates the needed operational records.
1. Keep SaaS authentication and operational authentication boundaries clear.

## Payment Flow

Flow:

1. User selects a plan.
2. App creates or references a checkout session.
3. User goes to Paddle checkout.
4. User returns through checkout return or cancel pages.
5. Paddle sends webhook events.
6. Webhook-confirmed payment updates local payment/billing state.
7. Verified payment can make a pending organization ready for provisioning.

Guardrails:

1. Webhook confirmation is authoritative.
1. Do not use return-page navigation as proof of payment.
1. Keep Paddle-specific details inside `saas/` payment/client service boundaries.

## Provisioning Flow

Flow:

1. Pending organization has completed required onboarding.
2. Payment is verified or owner-approved readiness is satisfied.
3. Provisioning job is queued or run.
4. Operational records are created or connected:

1. school group,
1. branch,
1. academic year,
1. initial operational user,
1. permissions/role context,
1. required setup defaults.

1. Provisioning status is updated.
2. Activation/access email may be sent.
3. User enters operational portal through `/login`.

Guardrails:

1. Keep provisioning idempotent where possible.
1. Do not mix school groups.
1. Do not skip platform owner visibility.

## Operational Login Flow

Flow:

1. User opens `/login`.
2. Credentials are verified.
3. Active-user status is checked.
4. Platform users route toward platform context.
5. Tenant users receive branch/year scope.
6. Session and scope cookies are set.
7. User lands on `/platform` or `/dashboard`.
8. Middleware enforces route permissions.

Guardrails:

1. Preserve platform vs tenant branching.
1. Preserve idle timeout behavior for tenant users.
1. Do not bypass permission middleware.

## Platform Owner Flow

Flow:

1. Platform owner logs in through `/login`.
2. Owner lands in platform context.
3. Owner uses `/platform` for organization context and owner/developer controls.
4. Owner uses SaaS admin pages for pending organizations, payments, and provisioning.

5. Owner uses `/platform/knowledge` to review KMS health.
6. Owner views/downloads the PDF through protected routes:

- `/platform/knowledge/booklet`
- `/platform/knowledge/booklet/download`

Guardrails:

- Platform developers are not owners.
- Owner-only pages must use existing owner access helpers.
- Do not expose KMS PDF through direct static links.

## Knowledge Management Flow

Flow:

1. Developer or Codex reads `docs/AI_PROJECT_CONTEXT.md`.
2. Developer reads master context, project state, engineering docs, relevant ADRs, and module history.
3. Approved change is implemented.
4. Knowledge Impact Assessment is completed.
5. Relevant docs are updated:

- master context,
- project state,
- change history,
- ADRs if needed,
- module history if needed,
- AI project context if needed,
- engineering docs if architecture/module/flow understanding changed.

1. PDF generator runs.
2. PDF snapshot and manifest are regenerated.
3. Knowledge Center checks manifest freshness and health.

Guardrails:

- Markdown remains source of truth.
- PDF is generated and must not be edited manually.
- App must not silently rewrite source docs.
- Regenerate button is not implemented yet.

## Human / AI Developer Onboarding Flow

Flow:

1. Read `docs/AI_PROJECT_CONTEXT.md`.
2. Read `docs/README.md`.
3. Read `docs/TIS_MASTER_CONTEXT.md`.
4. Read `docs/PROJECT_STATE.md`.
5. Read `docs/engineering/TIS_MODULE_MAP.md`.
6. Read `docs/engineering/REPOSITORY_ARCHITECTURE.md`.
7. Read `docs/engineering/USER_AND_SYSTEM_FLOWS.md`.
8. Read relevant ADRs and module history.
9. Inspect code with `rg` before editing.
10. Make scoped changes only.
11. Update KMS docs and regenerate the PDF if needed.
12. Report the KIA.

Before coding, inspect:

1. affected routes,
1. models and scope fields,
1. permission rules,
1. templates/forms,
1. tests for the touched module,
1. related docs/ADRs/history.

After coding, update:

1. `docs/CHANGE_HISTORY.md` for meaningful changes,
1. module history for area-specific changes,
1. ADRs for major decisions,
1. engineering docs when module maps, architecture, or flows change,
1. AI context when onboarding truth changes,
1. project state when priority/status changes.

# docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

# TIS Database Architecture Overview

This document explains the conceptual TIS data model. It is intentionally high level and does not list every field. Use `models.py`, `saas/models.py`, and tests for exact implementation details.

## Core Boundary Principle

TIS has three identity/data worlds that must never be casually mixed:

1. Platform data: platform owners, co-owners, developers, platform permissions, and cross-organization oversight.
1. SaaS account data: public signup, onboarding, billing, pending organizations, and payment/provisioning readiness.
1. Operational tenant data: provisioned school groups, branches, academic years, users, teachers, subjects, planning, timetable, calendar, and observations.

The safest mental model:

```
Platform Owner oversees many organizations.
SaaS account prepares an organization for subscription/provisioning.
Operational tenant data belongs to one provisioned school group/branch/year context.
```

## Platform Identities

Represents: Platform Owner, Co-Owner, and Platform Developer identities.

Ownership boundary: Platform identities can operate outside ordinary tenant scope only through approved platform workflows.

Must never be mixed:

1. Platform Developer must not become Platform Owner through permission drift.
1. Platform users must not be treated as normal tenant users unless an explicit context workflow establishes scope.

Related files:

1. `auth.py`
1. `models.py`
1. `permission_registry.py`
1. `main.py`

## SaaS Accounts

Represents: Public SaaS signup/login/account identities used before or alongside operational tenant access.

Ownership boundary: SaaS accounts own onboarding and billing/account state, not operational school records directly.

Must never be mixed:

1. SaaS account identity is not operational user identity.
1. SaaS account creation must not directly create live tenant data.

Related files:

- `saas/models.py`
- `saas/service.py`
- `saas/router.py`
- `docs/adr/0002-separate-saas-identity-and-operational-users.md`

## Pending Organizations

Represents: An organization moving through SaaS onboarding before operational provisioning.

Ownership boundary: Pending organization data belongs to the SaaS onboarding/provisioning pipeline.

Must never be mixed:

- Pending organization data is not the live school group until provisioning creates or connects operational records.

Related files:

- `saas/models.py`
- `saas/service.py`
- `saas/provisioning_service.py`

## Payment Records

Represents: Payment, billing, checkout, and provider-confirmed subscription state.

Ownership boundary: Payment state belongs to SaaS billing/subscription logic and should not be embedded directly into academic modules.

Must never be mixed:

- Checkout return navigation must not be treated as verified payment.
- Payment/provider details must not leak into operational teacher/planning/calendar logic.

Related files:

- `saas/payment_service.py`
- `saas/billing_service.py`
- `saas/paddle_client.py`
- `docs/adr/0003-paddle-payment-architecture.md`
- `docs/adr/0004-webhook-only-payment-confirmation.md`

## Provisioning Jobs

Represents: Controlled work that turns a ready pending organization into operational school structures.

Ownership boundary: Provisioning bridges SaaS data and operational tenant data, but only through explicit platform-owner-visible workflows.

Must never be mixed:

- Do not directly create operational tenant records from public signup or checkout return pages.

1. Do not create records under the wrong school group.

Related files:

1. `saas/provisioning_service.py`
1. `saas/router.py`
1. `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

## School Groups / Organizations

Represents: The top-level operational tenant boundary.

Ownership boundary: Most tenant operational data belongs to a school group directly or through branch/year relationships.

Must never be mixed:

1. Data from one school group must not appear in another school group's operational workflows.

Related files:

1. `models.py`
1. `main.py`

## Branches

Represents: Campuses or branches inside a school group.

Ownership boundary: Branches scope users, teachers, planning, timetable, academic calendar, branding, and reports.

Must never be mixed:

1. Branch-scoped data must not silently cross campuses.
1. Platform context switching must remain explicit.

Related files:

1. `models.py`
1. `main.py`
1. `ui_shell.py`

## Academic Years

Represents: The academic period used to scope operational records.

Ownership boundary: Planning, timetable, subjects, calendar, and related data may depend on active academic year.

Must never be mixed:

1. Do not copy or read current-year data into another year unless the workflow explicitly does so.

Related files:

1. `models.py`
1. `main.py`

- `year_copy.py`

## Operational Users

Represents: Users who work inside an operational school group/branch/year context.

Ownership boundary: Operational users belong to tenant structures and are governed by roles, permissions, branch scope, and active status.

Must never be mixed:

- Operational users are not SaaS accounts by default.
- Tenant users should not gain platform owner access.

Related files:

- `models.py`
- `auth.py`
- `routers/users.py`

## Roles And Permissions

Represents: Role packages, permission keys, platform developer permissions, and route/action authorization.

Ownership boundary: Permissions decide what a user can view or change in the current scope.

Must never be mixed:

- UI hiding is not enough; protected actions need route/service checks.
- Owner controls must remain outside developer-assignable permission drift.

Related files:

- `permission_registry.py`
- `role_permission_service.py`
- `authorization.py`
- `auth.py`

## Teachers

Represents: Teacher records, qualifications, capacity, workload, and staffing-relevant academic data.

Ownership boundary: Teacher records are tenant/branch/year-sensitive operational data.

Must never be mixed:

- Teacher data from one branch or school group must not appear in another tenant's planning/reporting.

Related files:

- `routers/teachers.py`
- `teacher_qualifications.py`
- `teacher_capacity.py`

- `models.py`

## Subjects

Represents: Subject catalog, colors, requirements, qualification relationships, planning and timetable dependencies.

Ownership boundary: Subjects are academic configuration data inside tenant/year context.

Must never be mixed:

- Subject changes can affect planning, timetable, teacher matching, and reports; update all affected docs/tests.

Related files:

- `routers/subjects.py`
- `subject_colors.py`
- `models.py`

## Sections / Classes

Represents: Class sections used by planning and timetabling.

Ownership boundary: Sections belong to operational branch/year structures.

Must never be mixed:

- Section structures should not cross branch/year boundaries accidentally.

Related files:

- `routers/planning.py`
- `models.py`

## Workforce Planning

Represents: Teacher assignments, homeroom ownership, capacity, workload, subject coverage, and staffing needs.

Ownership boundary: Planning is operational data tied to school group, branch, academic year, teachers, subjects, and sections.

Must never be mixed:

- Planning changes can affect dashboards, reports, timetable, and staffing decisions.

Related files:

- `routers/planning.py`
- `teacher_capacity.py`
- `homeroom_defaults.py`

## Timetable

Represents: Weekly lesson placement and timetable settings.

Ownership boundary: Timetable data depends on planning, teacher, subject, section, branch, and year context.

Must never be mixed:

1. Timetable edits must preserve scheduling constraints and scope.

Related files:

1. `routers/timetable.py`
1. `timetable_logic.py`

## Academic Calendar

Represents: Events, academic dates, responsibilities, exports, and calendar settings.

Ownership boundary: Calendar records are branch/year/tenant scoped.

Must never be mixed:

1. Calendar events for one tenant or branch must not appear in another.

Related files:

1. `routers/academic_calendar.py`
1. `templates/academic_calendar.html`

## Observations

Represents: Teacher observation records, feedback, scoring, evidence, and history.

Ownership boundary: Observation data is sensitive tenant operational data.

Must never be mixed:

1. Observation records must remain tenant-scoped and permission-protected.

Related files:

1. `routers/observations.py`
1. `templates/observation_*.html`

## Knowledge Management System Artifacts

Represents: Markdown docs, ADRs, module history, generated PDF, manifest, and Knowledge Center read-only status.

Ownership boundary: Markdown under `docs/` is source of truth. Generated artifacts under `static/docs/` are snapshots.

Must never be mixed:

1. The app must not silently rewrite Markdown source docs.
1. The PDF must not be edited manually.
1. Protected documentation access should go through owner-only routes.

Related files:

1. `docs/`
1. `scripts/generate_docs_pdf.py`

- `static/docs/`
- `knowledge_service.py`

## Tenant Isolation Rules

- Always know the current school group, branch, and academic year.
- Do not assume platform users have tenant scope.
- Do not assume SaaS accounts have operational records.
- Keep pending SaaS organization state separate from provisioned operational data.
- Preserve route permissions and service-level checks.
- Tests touching cross-tenant data should be treated as high value.

# docs/engineering/DEVELOPMENT_STANDARDS.md

# TIS Development Standards

These standards are mandatory for future human developers, Codex conversations, ChatGPT conversations, and technical reviewers working on TIS.

## Inspect Before Editing

Before changing files:

1. inspect the current code with `rg` and targeted file reads,
1. understand the affected route/service/template/model,
1. read relevant docs, ADRs, and module history,
1. check tests for the touched module,
1. check the current branch and working tree.

Do not code from memory when local context is available.

## Plan Before Implementation

For non-trivial work:

1. identify the affected modules,
1. identify tenant/permission/identity boundaries,
1. identify documentation updates,
1. identify tests or smoke checks,
1. keep the implementation path small.

Planning does not need to be long, but the risk boundaries must be clear.

## Keep Changes Small And Scoped

1. Implement only the approved request.
1. Avoid unrelated refactors.
1. Avoid formatting churn in unrelated files.
1. Preserve existing patterns unless there is a clear reason to change them.
1. Do not move code across modules as a side effect.

## Preserve Tenant Isolation

Tenant isolation is non-negotiable.

Always protect:

1. school group boundaries,
1. branch boundaries,
1. academic year scope,

- user role/permission scope,
- observation and teacher data sensitivity.

Any change that touches tenant scope should be treated as high risk.

## Preserve Login And Platform Owner Flows

Do not casually change:

- `/login`,
- session cookies,
- platform owner detection,
- platform developer permissions,
- `/platform`,
- `/platform/knowledge`,
- scope switching,
- access denied handling.

Platform developers are not platform owners unless existing owner logic explicitly treats them that way.

## Protect SaaS, Payment, And Provisioning

Do not casually change:

- `/saas/signup`,
- `/saas/login`,
- `/saas/account`,
- SaaS onboarding steps,
- Paddle checkout/client code,
- payment confirmation logic,
- billing state,
- provisioning readiness,
- provisioning jobs.

Webhook-confirmed payment is authoritative. Checkout return pages are not proof of payment.

## Database And Migration Rules

- Never modify `tis.db` unless explicitly approved.
- Never modify migrations casually.
- Do not change `models.py` without understanding migration/repair implications.
- Do not add schema changes without tests and documentation.
- Never use destructive database operations without explicit approval.

## Landing Website Rule

The public landing website is separate:

1. implementation: `tis-landing-website/`,
1. marketing docs: `docs/marketing/`,
1. ADRs: `0001`, `0007`.

Do not modify landing page code during backend or operational app tasks unless explicitly approved.

## Protected Documentation Rule

1. Do not expose generated PDFs through direct public UI links.
1. Use owner-protected routes for PDF access.
1. Markdown source docs are reviewed development artifacts.
1. The app must not silently rewrite Markdown source docs.

## Production Memory And Render Stability

Render memory is a hard production budget. Treat a 512 MB service as constrained and design routes, services, and assets accordingly.

Strict rules:

1. Do not load, parse, or cache complete large datasets at startup or on a normal first request when a scoped lookup, streaming parser, pagination, or bounded cache can be used.
1. Do not keep both raw and transformed copies of large JSON, CSV, Excel, PDF, image, or uploaded payloads in memory unless explicitly justified and validated.
1. Do not run duplicate template renders in production request paths. Diagnostic pre-renders must be removed or gated behind an explicit environment flag.
1. Do not emit stage-by-stage debug logs at warning level during normal traffic. Production diagnostics must be opt-in and should use appropriate log levels.
1. Filter by tenant, school group, branch, academic year, and permission scope before materializing query results.
1. Keep caches bounded by size or scope. Global caches for user-facing lookup data must be justified by measured memory impact.
1. Large exports should stream or build bounded artifacts; avoid repeatedly constructing large in-memory workbooks, PDFs, or payloads inside loops.
1. New static assets, videos, generated files, and location/reference datasets must be reviewed for repository size, runtime memory, and deployment impact.
1. Any feature that adds broad data aggregation, new startup work, large reference files, or route-level diagnostics must include a memory/performance smoke check before deployment.

Minimum validation for memory-sensitive changes:

1. run targeted tests for the touched module,
1. run a compile/import check for changed Python modules,
1. measure peak memory locally when a route or service parses large data,

1. review production logs after deployment for restarts, OOM symptoms, and unexpected warning spam.

## Commit And Push Rule

1. Do not commit unless explicitly requested.
1. Do not push unless explicitly requested.
1. Do not merge unless explicitly requested.
1. Final reports should describe changes and validation clearly.

## KMS Update Rule

Every meaningful implementation must update KMS if affected:

1. `docs/TIS_MASTER_CONTEXT.md`,
1. `docs/PROJECT_STATE.md`,
1. `docs/CHANGE_HISTORY.md`,
1. `docs/AI_PROJECT_CONTEXT.md`,
1. ADRs if major decisions changed,
1. module history for module-specific before/after state,
1. engineering docs if module maps, architecture, data model, standards, design philosophy, roadmap, or flows changed.

Regenerate the PDF if included docs changed.

## Knowledge Impact Assessment

Every final implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed.

## Validation Standards

Run the narrowest meaningful validation:

1. compile checks for Python scripts/modules,
1. targeted pytest tests when behavior changes,
1. route/template smoke checks when pages are touched,
1. PDF generation when docs change,
1. frontend checks when landing/frontend code changes.

If validation cannot be run, say why.

# docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md

# TIS UI/UX Design Philosophy

TIS should feel like a clean, premium academic SaaS platform. It should not feel like a generic admin dashboard, spreadsheet wrapper, or decorative marketing shell disconnected from real product value.

## Overall Product Identity

Design principles:

1. professional,
1. light,
1. trustworthy,
1. academic,
1. calm,
1. data-aware,
1. premium without being flashy.

The product serves serious academic operations. UI should reduce confusion and support confident decisions.

## Operational FastAPI App

The operational app should prioritize:

1. clarity,
1. role-based navigation,
1. data density,
1. fast scanning,
1. predictable controls,
1. strong permission boundaries,
1. branch/year context visibility.

Avoid:

1. generic admin clutter,
1. decorative cards that do not help workflows,
1. large marketing-style hero sections inside operational tools,
1. hiding important state behind vague labels,
1. layouts that make repeated operational use slow.

Operational pages should help users answer:

1. What am I looking at?
1. Which school/branch/year is active?
1. What needs action?

1. What can my role do here?
1. What changed after I saved?

## Platform Owner Console

The Platform Owner Console should feel like a controlled operations console.

Priorities:

1. global visibility,
1. owner/developer separation,
1. clear organization switching,
1. safe access to sensitive tools,
1. explicit status labels,
1. no accidental tenant context confusion.

Avoid:

1. broad controls without owner-only checks,
1. exposing platform tools to developers by appearance alone,
1. unclear organization/branch state.

## Knowledge Center

The Knowledge Center is an internal owner utility, not a marketing page.

Priorities:

1. KMS health,
1. source coverage,
1. freshness status,
1. protected PDF actions,
1. ADR/change/module visibility,
1. KIA policy reminder.

Avoid:

1. direct public static PDF links,
1. regenerate actions until approved,
1. app-side Markdown rewriting,
1. decorative presentation that hides status.

## SaaS Onboarding Pages

SaaS onboarding should feel guided, calm, and customer-facing.

Priorities:

1. clear next step,

- progress visibility,
- plain customer language,
- reassurance around setup and billing,
- no internal engineering terms.

Customer-facing language should avoid:

- tenant,
- provisioning,
- M1/M2/M3/M4/M5,
- schema,
- migration,
- internal role mechanics.

Use customer language such as:

- organization,
- school,
- branch or campus,
- setup,
- plan,
- billing,
- account,
- getting your workspace ready.

## Next.js Landing Website

The landing website should use premium storytelling and strong visual assets.

Priorities:

- clear problem/solution narrative,
- actual product credibility,
- school operations language,
- strong visual hierarchy,
- polished screenshots or generated assets when appropriate,
- conversion path into demo/signup.

Avoid:

- weak fake 3D visuals,
- generic SaaS gradients without product specificity,
- vague claims unsupported by product reality,
- internal terms like tenant/provisioning/milestones,

1. cluttered feature walls.

The landing page source of truth is `tis-landing-website/`, not legacy FastAPI landing files.

## Visual System Direction

Future design system should support:

1. consistent cards, tables, filters, status badges, and action buttons,
1. clear empty/loading/error states,
1. accessible contrast,
1. stable layouts across modules,
1. consistent icon and label patterns,
1. role-aware navigation,
1. responsive behavior without losing operational density.

## Tone And Copy

Internal app copy:

1. concise,
1. action-oriented,
1. operationally clear.

Customer-facing copy:

1. plain,
1. confident,
1. benefit-oriented,
1. free of internal implementation terms.

Platform owner copy:

1. precise,
1. status-driven,
1. explicit about access and risk.

# docs/engineering/PRODUCT_ROADMAP.md

# TIS Product Roadmap

This roadmap summarizes completed, current, next, and future product directions. It is not a release commitment; it is the current planning map for developers, owners, and reviewers.

## Completed

### SaaS Identity Foundation

Established SaaS account signup, login, account area, and the boundary between SaaS accounts, platform identities, and operational tenant users.

### Pending Organizations Zone

Created pending organization structures and owner-facing review concepts for organizations moving through SaaS onboarding.

### Plans And Billing Foundation

Added plan catalog, plan selection, billing status, checkout summary, checkout return/cancel, and payment/billing service boundaries.

### Paddle Payment Collection

Added Paddle-oriented payment architecture with provider logic isolated behind service/client modules.

### Tenant Provisioning Engine

Added provisioning workflows and jobs to create or connect operational school group/branch/year/user records from ready pending organizations.

### KMS Foundation

Created source-of-truth Markdown docs, change history, ADRs, module history, AI project context, generated PDF booklet, and manifest.

### Platform Owner Knowledge Center

Added protected owner-only Knowledge Center for KMS health, source freshness, manifest metadata, ADRs, module history, and protected PDF view/download.

### KMS v3.0 Phase 3A

Added engineering handbook foundation with module map, repository architecture, user/system flows, and onboarding structure.

## Current

### KMS v3.0 Enhancement

Current KMS work expands the engineering handbook with database architecture, development standards, UI/UX philosophy, roadmap, and stronger developer/AI guidance.

### Landing / Customer Experience Preparation

The product is preparing for stronger public customer journey work from landing page storytelling through SaaS signup, onboarding, billing, and operational access.

## Next

### Premium Landing Page Redesign

Improve public website quality, product storytelling, visuals, and conversion clarity while preserving the separate Next.js source-of-truth.

### Customer Journey From Landing To Signup

Clarify the path from public website to SaaS signup, plan selection, onboarding, checkout, and workspace readiness.

### SaaS Onboarding Refinement

Improve onboarding language, progress, validation, customer reassurance, and platform owner visibility.

### Paddle Live Configuration

Configure live Paddle behavior when the payment account and production readiness are approved.

### Subscription Lifecycle Management

Add clearer management for active subscriptions, renewal state, payment failures, and account lifecycle.

### Feature Gating By Plan

Introduce plan-based access control for advanced capabilities, future AI features, and subscription tiers.

### Customer Billing Portal

Provide customer-facing billing management if supported by payment architecture and approved product flow.

### Trial / Upgrade / Downgrade / Cancellation Workflows

Define and implement subscription lifecycle actions safely with provider and local billing state alignment.

## Future

### AI Academic Assistant

Support academic leaders with intelligent recommendations based on verified tenant data.

### AI Assessment Generation

Generate curriculum-aware assessments or question banks when curriculum/planning data is reliable enough.

### AI Observation Improvement Plans

Assist supervisors with improvement plans, follow-up suggestions, and structured feedback from observation records.

### AI Calendar Activity Planning

Suggest academic calendar activities, reminders, and coordination plans.

### AI Dashboards And Academic Analytics

Provide richer academic insight, trends, risks, and multi-branch intelligence.

### Multi-Branch Executive Dashboards

Give leadership cross-branch visibility into staffing, workload, observations, calendar, and planning health.

### Curriculum And Planning Workflows

Expand from operational staffing/planning toward curriculum-aware academic planning if approved.

### Reporting Improvements

Improve exports, dashboards, executive summaries, and decision-support reports.

### Mobile Or Portal Expansion

Explore teacher/mobile/parent-style portals only if later approved and aligned with product strategy.

## Roadmap Guardrails

1. Do not build AI features before data quality, permissions, and plan boundaries are clear.
1. Do not build subscription lifecycle actions before payment provider behavior is verified.
1. Do not redesign landing/customer journey without preserving the Next.js boundary.
1. Do not expose internal terms to customers.
1. Update KMS docs and roadmap after meaningful roadmap changes.

# docs/engineering/REJECTED_DECISIONS.md

# TIS Rejected Architectural Decisions

This document records significant ideas that were considered and rejected. It helps future developers and AI assistants avoid reopening old decisions without new evidence.

Rejected decisions are not failures. They are part of the design record.

## Single Application For Landing And Portal

Date: 2026-06-26

Context: TIS needs both a public marketing site and a protected operational app. The public site needs premium storytelling and conversion design; the app needs authenticated academic operations.

Proposed solution: Use the FastAPI/Jinja app for both the public landing website and the operational portal.

Why rejected: The two surfaces have different runtimes, audiences, design needs, deployment concerns, and risk profiles. Mixing them would make backend/operational tasks more likely to accidentally change the public website.

Chosen solution: Keep `tis-landing-website/` as a separate Next.js public landing website and keep the FastAPI app as the operational portal.

Long-term consequences: Developers must respect the landing/portal boundary. Landing changes require explicit approval and must use the Next.js source of truth.

## Stripe Instead Of Paddle

Date: 2026-06-26

Context: TIS needs subscription payment collection, billing state, and future subscription lifecycle workflows.

Proposed solution: Use Stripe as the payment provider.

Why rejected: The current approved architecture is Paddle-oriented. Changing providers would reopen checkout, webhook, billing, tax/compliance, customer portal, and subscription lifecycle assumptions.

Chosen solution: Use Paddle behind service/client boundaries in `saas/`.

Long-term consequences: Provider-specific behavior must remain isolated. A future provider change would require a new ADR and careful migration plan.

## Immediate Tenant Provisioning

Date: 2026-06-26

Context: Public signup and SaaS onboarding collect organization data before a school is ready for live operational access.

Proposed solution: Create operational school group, branch, user, and academic records immediately after signup or checkout return.

Why rejected: Immediate provisioning risks creating live tenant data before payment is verified, before onboarding is complete, or before platform owner review. Checkout return pages are not authoritative payment confirmation.

Chosen solution: Delay tenant provisioning until payment/readiness is verified and use platform-owner-visible provisioning jobs.

Long-term consequences: There are more states between signup and operational login, but tenant data is safer and provisioning can be reviewed/retried.

## Public Documentation Access

Date: 2026-06-26

Context: The KMS generates a PDF under `static/docs/`, but the content includes internal architecture, roadmap, and operational guidance.

Proposed solution: Link directly to the generated PDF through public static paths.

Why rejected: Internal documentation should not be exposed as a public asset. Static file serving is not authorization-aware.

Chosen solution: Expose the PDF through protected Platform Owner routes only.

Long-term consequences: The UI must not link directly to `/static/docs/...`. Future storage may move generated docs outside public static storage.

## Documentation Without ADRs

Date: 2026-06-26

Context: TIS decisions span SaaS identity, payments, provisioning, landing architecture, and KMS governance.

Proposed solution: Keep only a master context and change history.

Why rejected: Chronological history does not explain major decision tradeoffs deeply enough. Future developers would repeatedly reopen settled architecture questions.

Chosen solution: Maintain ADRs under `docs/adr/` for major accepted decisions and this rejected-decision record for important alternatives.

Long-term consequences: Major decisions require explicit traceability. ADRs and rejected decisions must be updated when architecture changes.

## Weak Generic Landing Page Design

Date: 2026-06-26

Context: The public landing website is the first impression for schools evaluating TIS.

Proposed solution: Use a generic SaaS layout with vague claims, weak fake 3D visuals, or internal implementation language.

Why rejected: TIS needs credibility with school leaders. Generic visuals and internal language reduce trust and do not communicate academic operations value.

Chosen solution: Use premium storytelling, strong visual assets, clear customer language, and the separate Next.js landing visual system.

Long-term consequences: Landing work should be deliberate, visually strong, and customer-facing. Avoid terms like tenant, provisioning, and internal milestone labels.

## Mixing SaaS Identities With Operational Users

Date: 2026-06-26

Context: TIS has public SaaS accounts, operational tenant users, and platform identities.

Proposed solution: Use one identity concept for signup, billing, tenant access, and platform administration.

Why rejected: This would weaken security and make it easy to confuse pending accounts with live operational users or platform owners.

Chosen solution: Keep SaaS account identity, operational tenant identity, and platform identity distinct.

Long-term consequences: Developers must always ask which identity world a change belongs to. Provisioning bridges worlds only through approved flows.

## App-Side Automatic Documentation Rewriting

Date: 2026-06-26

Context: The KMS requires docs to remain current after meaningful implementation work.

Proposed solution: Let the running app automatically rewrite Markdown docs when it detects staleness.

Why rejected: Source-of-truth docs must be reviewed development artifacts. Silent rewriting would damage reviewability and could introduce incorrect historical records.

Chosen solution: Developers or AI assistants update Markdown docs as part of approved implementation work. The app may detect freshness and expose status, but not rewrite source docs.

Long-term consequences: Knowledge Impact Assessment remains a required human/AI workflow step. Future regenerate actions may rebuild PDFs only from reviewed Markdown.

## Uncontrolled Regenerate Button

Date: 2026-06-26

Context: The Knowledge Center can show whether the PDF snapshot is current or stale.

Proposed solution: Add an immediate button that regenerates docs and/or source files from the app.

Why rejected: Phase 2C is read-only. Runtime regeneration raises deployment persistence, audit, concurrency, and source-control questions.

Chosen solution: Do not add regenerate behavior until separately approved. Keep source docs updated through reviewed development work.

Long-term consequences: A future regenerate action must be explicit, owner-only, audited where appropriate, and must not rewrite Markdown source docs.

# docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md

# TIS Visual Documentation Guide

This guide defines how visual documentation should evolve. No screenshots are required yet, but future screenshots and diagrams should follow this structure.

## Purpose

Visual documentation should help developers, reviewers, owners, and future AI assistants understand TIS screens, workflows, data relationships, and design expectations.

Visual docs should not replace Markdown source docs. They should support them.

## Future Storage Location

Recommended future location:

```
docs/visuals/
  screenshots/
  diagrams/
  flows/
```

Recommended generated/reference output:

```
static/docs/visuals/
```

Do not store sensitive production data in screenshots.

## Screenshot Standards

Screenshots should:

1. use clean test/demo data,
1. avoid private student/teacher/customer information,
1. show the full relevant viewport or focused workflow area,
1. include enough context to identify branch/year/page state,
1. avoid browser chrome unless needed,
1. be updated when UI behavior or layout meaningfully changes.

Preferred formats:

1. PNG for UI screenshots,
1. SVG or PNG for diagrams,
1. PDF only for exported documentation snapshots.

## Naming Convention

Use stable, descriptive names:

```
YYYY-MM-DD_module_screen_state.png
YYYY-MM-DD_flow_name_step_number.png
YYYY-MM-DD_architecture_diagram_name.svg
```

Examples:

```
2026-06-26_platform-console_owner-view.png
2026-06-26_knowledge-center_current-status.png
2026-06-26_saas-onboarding_step-organization.png
```

## Update Policy

Update visual docs when:

1. a screen layout meaningfully changes,
1. workflow steps change,
1. a module gains a new major view,
1. navigation or role visibility changes,
1. a diagram no longer reflects architecture.

Visual doc updates should be mentioned in the Knowledge Impact Assessment.

## Future Diagram Strategy

Future diagrams should cover:

1. identity separation,
1. SaaS onboarding to provisioning,
1. payment webhook confirmation,
1. tenant isolation,
1. operational data model,
1. KMS source-to-PDF-to-Knowledge-Center flow,
1. landing-to-signup customer journey.

Diagrams should be simple and versioned through Markdown references.

## Planned Visual Areas

### Platform Console

Future visuals:

1. owner account controls,
1. developer management,
1. organization/branch context switching.

### Knowledge Center

Future visuals:

1. healthy status,
1. stale status,
1. source document table,
1. KIA panel.

## Dashboard

Future visuals:

1. operational dashboard overview,
1. staffing/report widgets,
1. scope context display.

## Teachers

Future visuals:

1. teacher list,
1. teacher edit/profile,
1. qualification/capacity views.

## Workforce Planning

Future visuals:

1. planning table,
1. section ownership,
1. workload/capacity states.

## Academic Calendar

Future visuals:

1. calendar view,
1. event editing,
1. exports or responsibility states.

## SaaS Onboarding

Future visuals:

1. signup,
1. organization step,
1. contacts,
1. branches,
1. academic setup,
1. review.

## Billing

Future visuals:

1. plan selection,
1. checkout summary,

1. billing status,
1. payment pending/verified states.

### Landing Page

Future visuals:

1. hero,
1. problem/solution section,
1. product capability section,
1. signup/demo conversion paths.

### Reports

Future visuals:

1. allocation report,
1. dashboards,
1. export states.

## Guardrails

1. Do not include sensitive real data.
1. Do not let visual docs become the only source of truth.
1. Do not update landing visuals during backend tasks unless explicitly approved.
1. Keep visual docs aligned with KMS change history.

# docs/engineering/AI_OPTIMIZATION_GUIDE.md

# TIS AI Optimization Guide

This is the definitive onboarding guide for future AI assistants working on TIS.

## Preferred Onboarding Order

For any new AI coding conversation:

1. Read `docs/AI_PROJECT_CONTEXT.md`.
2. Read `docs/README.md`.
3. Read `docs/PROJECT_STATE.md`.
4. Read `docs/TIS_MASTER_CONTEXT.md`.
5. Read `docs/engineering/README.md`.
6. Read task-specific engineering docs.
7. Read relevant ADRs.
8. Read relevant module history.
9. Inspect code with `rg` before editing.

## Choosing Task-Specific Docs

Use this map:

1. SaaS signup/login/account: `docs/history/saas-onboarding/`, ADR 0002.
1. Billing/payment/Paddle: `docs/history/subscriptions/`, ADR 0003, ADR 0004.
1. Provisioning: `docs/history/provisioning/`, ADR 0005.
1. Landing page: `docs/marketing/`, ADR 0001, ADR 0007.
1. Platform owner/Knowledge Center: `docs/history/platform-knowledge/`.
1. Workforce planning/timetable/teachers/subjects: `docs/history/workforce-planning/`.
1. Calendar: `docs/history/academic-calendar/`.
1. Architecture/data model: `docs/engineering/REPOSITORY_ARCHITECTURE.md`, `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`.
1. Design/UI: `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`.
1. Standards: `docs/engineering/DEVELOPMENT_STANDARDS.md`.
1. Roadmap: `docs/engineering/PRODUCT_ROADMAP.md`.

## How To Interpret ADRs

ADRs explain accepted long-term decisions.

When an ADR applies:

1. treat it as design authority,
1. do not reverse it casually,

1. if a change conflicts with it, propose a new ADR or mark it superseded only with approval,
1. link related docs/history.

## How To Use Module History

Module history preserves deeper before/after context.

Before changing a module:

1. read the module history folder,
1. identify the previous documented state,
1. update it if the module's meaning changes,
1. do not replace chronological `CHANGE_HISTORY.md` with module history.

## How To Complete KIA

Every final response must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

If meaningful behavior, architecture, workflow, roadmap, design, data model, or product state changes, documentation probably changed too.

## Avoiding Architectural Regressions

Do not:

1. merge SaaS account identity with operational users,
1. treat checkout return as verified payment,
1. provision tenants immediately from public signup,
1. expose internal docs publicly,
1. let platform developers act as platform owners,
1. modify landing code during backend tasks,
1. rewrite source docs from the running app,
1. change database/migrations without explicit approval.

## Preserving Tenant Isolation

Always identify:

1. school group,
1. branch,
1. academic year,
1. current user,

- user type,
- role/permissions,
- whether the user is platform, tenant, or SaaS account.

If scope is unclear, inspect code and tests before editing.

## Distinguishing Operational vs SaaS Code

Operational app:

- root FastAPI files,
- `routers/`,
- `templates/` operational templates,
- `models.py`,
- branch/year/teacher/planning/calendar/observation workflows.

SaaS:

- `saas/`,
- `templates/saas/`,
- signup/login/account/onboarding/billing/provisioning readiness.

Landing:

- `tis-landing-website/`,
- `docs/marketing/`.

KMS:

- `docs/`,
- `scripts/generate_docs_pdf.py`,
- `knowledge_service.py`,
- `templates/platform_knowledge_center.html`,
- `static/docs/`.

## Prompt Engineering Recommendations

Future prompts should specify:

- implementation vs planning,
- allowed files,
- forbidden files,
- whether docs must be updated,
- whether PDF regeneration is required,
- whether routes/app behavior can change,
- whether database/migrations are in scope,

1. whether landing is in scope.

Good prompt pattern:

```
Implement [specific goal].
Allowed files: [...]
Do not modify: [...]
Update KMS docs and regenerate PDF if docs change.
Run: [...]
Report KIA.
```

## AI Readiness Review

Current strengths:

1. AI context exists.
1. Engineering docs cover modules, repository, flows, database, standards, UI/UX, roadmap, rejected decisions, governance, and visual docs.
1. ADRs and module history preserve decision context.
1. PDF/manifest provide generated snapshot and freshness metadata.

Remaining gaps for future KMS improvements:

1. More screenshots and diagrams are needed.
1. More module-specific deep docs may be useful for timetable, observations, permissions, and dashboard/reporting.
1. Test strategy documentation could be expanded.
1. Deployment/runbook documentation could be expanded.
1. API/route inventory could be generated or manually documented later.
1. Data migration history could be summarized more explicitly.

Do not implement these gaps unless a future phase approves them.

# docs/engineering/PROJECT_GOVERNANCE.md

# TIS Project Governance

This document defines how TIS engineering decisions, approvals, documentation, and quality gates should be governed.

## Ownership Model

Product/project ownership: Defines business direction, roadmap priorities, customer experience, and approval for major changes.

Platform Owner: Owns platform-level operations, platform accounts, provisioning oversight, and KMS visibility.

Engineering implementer: Human developer, Codex, or ChatGPT-assisted coding session responsible for scoped implementation and validation.

Reviewer: Reviews implementation, KMS updates, risk boundaries, and validation results.

## Architectural Authority

Architectural authority comes from:

1. approved user direction,
1. ADRs,
1. `docs/TIS_MASTER_CONTEXT.md`,
1. engineering handbook docs,
1. module history,
1. existing code patterns.

If these conflict, pause and clarify before making irreversible changes.

## Approval Workflow

Major work should follow:

1. clarify objective and scope,
2. identify allowed/forbidden files,
3. inspect code/docs,
4. implement scoped change,
5. update KMS,
6. run validation,
7. report KIA,
8. wait for review before broader phases.

## Documentation Authority

Markdown under `docs/` is the source of truth.

Generated artifacts:

- PDF booklet,
- manifest,
- future screenshots/diagrams.

Generated artifacts support the docs; they do not replace them.

## Coding Workflow

Default workflow:

- inspect before editing,
- prefer existing patterns,
- keep changes small,
- preserve tenant isolation,
- preserve identity boundaries,
- avoid unrelated rewrites,
- run targeted validation,
- update KMS when meaningful.

## Review Expectations

Review should check:

- behavior matches scope,
- no forbidden files changed,
- tenant isolation preserved,
- permissions preserved,
- SaaS/payment/provisioning untouched unless approved,
- landing untouched unless approved,
- docs/KIA complete,
- validation credible.

## Branch Strategy

Current assumption:

- active development branch: `dev`,
- production/live branch is separate and must be confirmed before deployment.

Rules:

- do not push unless requested,
- do not commit unless requested,
- do not merge unless requested,

1. preserve unrelated local changes.

## Release Philosophy

Prefer:

1. small reviewable increments,
1. documentation updated with implementation,
1. explicit milestone boundaries,
1. staged rollout for risky systems,
1. no hidden operational changes.

High-risk areas:

1. authentication,
1. platform owner access,
1. tenant isolation,
1. payment,
1. provisioning,
1. database/migrations,
1. landing conversion path.

## Quality Gates

Before final report:

1. run relevant compile/tests/smoke checks,
1. verify generated docs if docs changed,
1. inspect working-tree scope,
1. mention validation gaps if any.

## Documentation Gates

Every meaningful implementation must answer:

1. Did master context change?
1. Did project state change?
1. Did change history change?
1. Is an ADR needed?
1. Is module history needed?
1. Did AI project context need updating?
1. Did engineering docs need updating?
1. Was PDF regenerated?

## KIA Requirement

Final reports must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

## Engineering Decision Traceability

Use this traceability model:

1. ADR: why a major decision was accepted.
1. Rejected decisions: why significant alternatives were not chosen.
1. CHANGE_HISTORY: chronological summary of meaningful changes.
1. Module history: deeper before/after context for one area.
1. Master context: durable current truth.
1. Project state: current status, priority, known issues, next work.
1. AI project context: compact first-read AI onboarding.
1. Engineering handbook: module, architecture, flow, standards, data, UI, roadmap, governance knowledge.
1. Generated PDF: snapshot for review/reference.
1. Manifest: generated metadata and source freshness basis.

When to update each:

1. Update ADRs for major decisions.
1. Update rejected decisions when a significant alternative is intentionally declined.
1. Update change history for meaningful changes.
1. Update module history for area-specific evolution.
1. Update master context for durable current truth.
1. Update project state for current status and next work.
1. Update AI context when first-read onboarding truth changes.
1. Update engineering docs when developer understanding changes.
1. Regenerate PDF/manifest when included docs change.

# docs/engineering/KNOWLEDGE_LIFECYCLE.md

# TIS Knowledge Lifecycle

This document defines how the TIS Knowledge Management System evolves with the software from planning through deployment and maintenance.

## Core Principle

Documentation evolves with implementation. Markdown is the source of truth. The PDF and manifest are generated snapshots. The Knowledge Center is the owner-facing status surface.

No meaningful implementation is complete until the Knowledge Impact Assessment is complete.

## Documentation Lifecycle

1. A task is proposed.
2. Relevant docs, ADRs, and module history are read.
3. The likely documentation impact is identified.
4. Implementation happens within approved scope.
5. KIA is completed.
6. Relevant Markdown docs are updated.
7. ADRs are added or updated if decisions changed.
8. Module history is updated if a module's documented state changed.
9. PDF and manifest are regenerated if included docs changed.
10. Final report describes the knowledge impact.

## Engineering Lifecycle

1. Inspect before editing.
2. Plan the smallest safe implementation.
3. Preserve tenant, identity, permission, SaaS, payment, provisioning, and landing boundaries.
4. Implement scoped changes.
5. Validate with focused checks.
6. Update KMS.
7. Report KIA.
8. Wait for review, commit, push, or deployment approval.

## Approval Lifecycle

1. Owner defines goal and constraints.
2. Implementer confirms scope.
3. Implementer makes only approved changes.

4. Reviewer checks behavior, scope, docs, validation, and KIA.
5. Commit/push/deployment happens only after explicit approval.

## Review Lifecycle

Review should verify:

- no forbidden files changed,
- tenant isolation remains intact,
- SaaS/payment/provisioning boundaries are preserved,
- landing code remains untouched unless approved,
- database/migrations are untouched unless approved,
- KMS docs and generated artifacts are current,
- KIA is complete.

## Release Lifecycle

Before release:

1. Confirm branch and deployment target.
2. Confirm tests/checks.
3. Confirm docs/PDF/manifest are current.
4. Confirm change history and project state.
5. Confirm release notes or owner communication if needed.
6. Deploy only through approved process.

## Maintenance Lifecycle

Maintenance includes:

- periodic KMS freshness review,
- periodic ADR/rejected-decision review,
- module history cleanup for discoverability,
- PDF readability review,
- Knowledge Center health checks,
- updating roadmap/current state after milestones.

## Planning To Production

Planning creates the first knowledge obligation. Production closes it.

Expected path:

```
Idea / task
  -> scope and constraints
  -> docs and code inspection
  -> implementation
  -> validation
```

```
-> KIA
-> docs/history/ADR updates
-> PDF/manifest regeneration
-> review
-> commit/push
-> deployment
-> post-deployment state update if needed
```

## Guardrails

1. The app must not silently rewrite Markdown docs.
1. Generated PDFs must not be edited manually.
1. Regeneration is artifact generation, not source editing.
1. KMS update work must remain reviewable.

# docs/engineering/DOCUMENTATION_AUTOMATION.md

# TIS Documentation Automation

This document defines current and future automation expectations for the TIS KMS.

## Current Automation

Current approved automation:

1. `scripts/generate_docs_pdf.py` reads approved Markdown sources.
1. It generates `static/docs/TIS_Project_Reference_Booklet.pdf`.
1. It generates `static/docs/docs_manifest.json`.
1. The manifest records documentation version, branch, commit SHA, source paths, mtimes, sizes, and hashes.
1. The Knowledge Center reads the manifest and checks freshness.

Current automation does not:

1. rewrite Markdown docs,
1. create ADRs,
1. create module history entries,
1. decide KIA outcomes,
1. commit or push changes.

## When Documentation Must Be Updated

Mandatory updates are required when a task changes:

1. architecture,
1. data model,
1. tenant isolation,
1. identity boundaries,
1. SaaS/account flows,
1. billing/payment/provisioning,
1. operational modules,
1. platform owner behavior,
1. UI/UX philosophy,
1. landing strategy,
1. roadmap,
1. governance,
1. development standards,
1. KMS behavior.

Optional updates may be skipped for:

1. typo-only code comments,
1. internal refactors with no behavior, architecture, flow, or ownership change,
1. generated files that do not affect source truth.

Even when docs are skipped, KIA must explain why.

## Regeneration Command

Use:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

Validation:

```
.\.venv\Scripts\python.exe -m py_compile scripts\generate_docs_pdf.py
```

## Manifest Lifecycle

The manifest is generated every time the PDF is regenerated.

It should be treated as:

1. generated metadata,
1. source freshness basis,
1. Knowledge Center input,
1. not the source of truth.

If included Markdown docs change and the manifest is not regenerated, the Knowledge Center should show stale status.

## PDF Lifecycle

The PDF is:

1. a generated snapshot,
1. owner reference material,
1. a reviewable artifact,
1. not manually edited.

Regenerate after included Markdown docs change.

## AI Context Lifecycle

Update `docs/AI_PROJECT_CONTEXT.md` when:

1. first-read onboarding changes,
1. current priority changes,
1. critical rules change,
1. new major docs are added,
1. architecture/identity/flow truth changes.

Do not overload AI context with every detail. It is a compact entry point.

## Handbook Lifecycle

Update engineering handbook docs when:

1. module boundaries change,
1. repository ownership changes,
1. user/system flows change,
1. database concepts change,
1. standards change,
1. design philosophy changes,
1. roadmap changes,
1. governance changes.

## Module History Lifecycle

Update module history when:

1. a specific module's previous documented state should be preserved,
1. a feature area changes meaningfully,
1. before/after context matters for future implementers.

## ADR Lifecycle

Create or update ADRs when:

1. major accepted decisions are made,
1. accepted decisions are superseded,
1. architecture or product boundaries change.

Use rejected decisions when important alternatives are intentionally declined.

## Mandatory vs Optional Summary

Mandatory:

1. KIA in final report,
1. docs update for meaningful changes,
1. change history for meaningful changes,
1. PDF/manifest regeneration when included docs change.

Conditional:

1. ADR for major decisions,
1. module history for area-specific state changes,
1. AI context for first-read truth changes,

1. engineering docs for handbook-level knowledge changes.

Optional:

1. visual docs until screenshots/diagrams are approved,
1. future CI/hooks/search automation until approved.

# docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md

# TIS Knowledge Impact Assessment Standard

The Knowledge Impact Assessment (KIA) is a required engineering standard. It confirms whether a task changed project knowledge and whether the KMS was kept current.

## Required KIA Fields

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

## Decision Tree

Ask:

1. Did behavior change?
2. Did architecture change?
3. Did a workflow change?
4. Did a module boundary change?
5. Did data ownership or tenant scope change?
6. Did SaaS/payment/provisioning change?
7. Did UI/UX or customer language change?
8. Did roadmap or current state change?
9. Did development standards/governance change?
10. Would a future developer or AI assistant need to know this?

If any answer is yes, knowledge impact is probably yes.

## Checklist

For knowledge-impacting work:

1. update relevant source docs,
1. update `docs/CHANGE_HISTORY.md`,
1. update module history if module state changed,
1. update ADRs if major decisions changed,
1. update rejected decisions if an important alternative was declined,
1. update AI project context if first-read truth changed,
1. regenerate PDF/manifest,

1. report validation.

## Examples

### Example: Docs Updated

```
Knowledge impact: Yes
Docs updated:
- docs/TIS_MASTER_CONTEXT.md
- docs/CHANGE_HISTORY.md
- docs/history/provisioning/...
Change history updated: Yes
ADR needed: No
Module history updated: Yes
PDF regenerated: Yes
AI project context updated: No
Reason if not updated: AI onboarding truth did not change.
```

### Example: No Docs Needed

```
Knowledge impact: No
Docs updated: None
Change history updated: No
ADR needed: No
Module history updated: No
PDF regenerated: No
AI project context updated: No
Reason if not updated: Change was limited to a typo in an internal variable name and
did not affect behavior, architecture, workflow, roadmap, or developer knowledge.
```

## Approval Expectations

Reviewers should reject final reports that omit KIA.

If docs were skipped, the reason must be specific. "Not needed" is not enough.

## Failure Cases

KIA fails when:

1. final report omits it,
1. meaningful behavior changed but docs did not,
1. docs changed but PDF/manifest were not regenerated,
1. ADR-worthy decisions were hidden in code changes,
1. module state changed without module history,
1. AI context became stale after major onboarding truth changed.

## Developer Responsibilities

Developers must:

1. assess KIA honestly,
1. update docs in the same task,
1. keep generated artifacts current,
1. avoid unrelated documentation churn,
1. preserve historical truth.

## AI Responsibilities

AI assistants must:

1. read relevant docs before coding,
1. identify likely KMS impact,
1. not invent history,
1. not silently skip docs,
1. not commit or push unless requested,
1. include KIA in every implementation final report.

# docs/engineering/SELF_EVOLVING_WORKFLOW.md

# TIS Self-Evolving Workflow

This is the official engineering workflow for keeping TIS software and knowledge synchronized.

## Workflow

```
Task
  -> Implementation
  -> Validation
  -> Knowledge Impact Assessment
  -> Documentation Updates
  -> ADR if required
  -> Module History if required
  -> Regenerate PDF
  -> Regenerate Manifest
  -> Review
  -> Commit
  -> Push
  -> Deployment
```

## Task

Clarify:

1. objective,
1. allowed files,
1. forbidden files,
1. validation requirements,
1. documentation requirements,
1. commit/push/deployment instructions.

## Implementation

Implement only the approved scope.

Protect:

1. tenant isolation,
1. identity boundaries,
1. payment/provisioning correctness,
1. owner-only controls,
1. landing boundaries,
1. database/migration safety.

## Validation

Run focused checks:

1. compile checks,
1. relevant tests,

1. route/template smoke checks,
1. PDF generation for docs,
1. frontend checks if frontend is in scope.

## Knowledge Impact Assessment

Complete KIA before final report.

Do not treat documentation as optional when knowledge changed.

## Documentation Updates

Update relevant docs:

1. master context,
1. project state,
1. AI context,
1. engineering docs,
1. change history,
1. module history,
1. ADRs/rejected decisions.

## ADR If Required

Create/update ADR when architecture or product boundaries change.

## Module History If Required

Update module history when area-specific before/after context matters.

## Regenerate PDF And Manifest

If included Markdown docs changed:

```
.\.venv\Scripts\python.exe scripts\generate_docs_pdf.py
```

## Review

Reviewers check:

1. scope,
1. behavior,
1. tests,
1. KMS updates,
1. generated artifacts,
1. KIA.

## Commit / Push / Deployment

These are explicit actions, not assumptions.

1. Commit only when requested.
1. Push only when requested.
1. Deploy only through approved process.

## Why This Is Self-Evolving

The workflow makes knowledge updates part of engineering completion. The system evolves because every meaningful change carries its context, history, decisions, and generated reference forward.

# docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md

# TIS Documentation Dependency Map

This document explains how KMS documents relate to each other and how changes propagate.

## Dependency Model

```
Master Context
  -> Project State
  -> Engineering Handbook
  -> AI Project Context
  -> ADR
  -> History
  -> Knowledge Center
  -> Manifest
  -> PDF
```

The arrows show common reading and propagation flow. They are not strict ownership hierarchy.

## Master Context

Purpose: Durable product, architecture, workflow, roadmap, and critical-rule truth.

Update when:

1. long-term architecture changes,
1. product vision changes,
1. major workflow changes,
1. critical rules change,
1. KMS structure changes.

## Project State

Purpose: Current branch, priority, milestone, known issue, and next-work truth.

Update when:

1. active priority changes,
1. phase status changes,
1. known issues change,
1. next planned work changes.

## Engineering Handbook

Purpose: Developer-facing deep knowledge: modules, repository, flows, database, standards, design, roadmap, governance, AI guidance, rejected decisions.

Update when:

1. developer understanding changes,
1. module maps change,

1 architecture or data ownership changes,

1 standards/governance changes,

1 future onboarding would be incomplete.

## AI Project Context

Purpose: Compact first-read file for future AI coding conversations.

Update when:

1 first-read onboarding changes,

1 major docs are added,

1 current priority changes,

1 critical boundaries change.

## ADR

Purpose: Accepted major decision record.

Update when:

1 a major architectural/product decision is made,

1 a prior decision is superseded,

1 a decision needs formal traceability.

## Rejected Decisions

Purpose: Record important alternatives that were declined.

Update when:

1 future teams are likely to reconsider a rejected path,

1 a rejected alternative explains the chosen architecture.

## History

Purpose: Chronological and module-specific evolution.

Update when:

1 meaningful change occurs,

1 module before/after state matters,

1 project knowledge changes.

## Knowledge Center

Purpose: Owner-facing app view of KMS health, freshness, and protected PDF access.

Update when:

1 KMS metadata behavior changes,

1. Knowledge Center UI/status behavior changes,
1. protected documentation access changes.

## Manifest

Purpose: Generated metadata for PDF and source freshness.

Update when:

1. PDF is regenerated.

Do not edit manually.

## PDF

Purpose: Generated handbook snapshot.

Update when:

1. included Markdown docs change.

Do not edit manually.

## Propagation Examples

Feature changes workflow:

```
Implementation -> CHANGE_HISTORY -> module history -> master/project state if needed
-> PDF/manifest
```

Architecture decision workflow:

```
Decision -> ADR -> rejected decision if relevant -> master context -> engineering
handbook -> change history -> PDF/manifest
```

Roadmap change workflow:

```
Roadmap -> PRODUCT_ROADMAP -> PROJECT_STATE -> AI_PROJECT_CONTEXT if first-read truth
changed -> PDF/manifest
```

KMS structure change workflow:

```
KMS docs -> README -> MASTER_CONTEXT -> AI_PROJECT_CONTEXT -> CHANGE_HISTORY ->
PDF/manifest
```

# docs/engineering/AI_CODING_WORKFLOW.md

# TIS AI Coding Workflow

This guide defines how future AI assistants should work inside TIS.

## Planning Before Coding

AI assistants must:

1. read `docs/AI_PROJECT_CONTEXT.md`,
1. read relevant engineering docs,
1. read relevant ADRs/module history,
1. inspect the codebase with `rg`,
1. identify allowed and forbidden files,
1. identify likely KMS impact,
1. avoid starting from assumptions.

## Implementation Review Before Editing

Before editing:

1. confirm the module boundary,
1. confirm tenant/identity/payment/provisioning risk,
1. confirm tests or validation,
1. confirm docs to update,
1. confirm that app, SaaS, landing, database, or routes are in scope before touching them.

## Implementation

During implementation:

1. keep changes narrow,
1. follow existing patterns,
1. avoid unrelated rewrites,
1. use `apply_patch` for manual edits,
1. do not commit or push,
1. do not modify `tis.db` unless explicitly approved.

## Validation Expectations

Run appropriate validation:

1. compile checks for Python scripts/modules,
1. targeted tests for behavior,

1. PDF generation for docs,
1. template/route smoke checks for UI routes,
1. frontend checks for landing/frontend tasks.

If validation cannot be run, state why.

## Documentation Updates

AI assistants must update KMS when knowledge changes:

1. change history,
1. project state,
1. master context,
1. AI context,
1. engineering docs,
1. ADRs/rejected decisions,
1. module history.

## KIA

Every final response must include KIA.

Do not omit KIA because the task "felt small." Assess it.

## Commit Strategy

AI assistants must not commit unless the user explicitly asks.

When committing is approved:

1. summarize scope,
1. ensure generated docs are current,
1. avoid unrelated changes,
1. use clear commit messages.

## Push Strategy

AI assistants must not push unless explicitly asked.

Before pushing:

1. confirm branch,
1. confirm tests/checks,
1. confirm KMS artifacts,
1. confirm no forbidden files changed.

## Deployment Strategy

AI assistants must not deploy unless explicitly asked.

Before deployment:

1. confirm production branch/environment,
1. confirm database/migration implications,
1. confirm SaaS/payment/provisioning risks,
1. confirm KMS state,
1. confirm rollback or recovery expectations.

## Final Response Pattern

Include:

1. files changed,
1. behavior changed or not,
1. validation,
1. known issues,
1. KIA,
1. no vague claims about tests that were not run.

# docs/engineering/FUTURE_AUTOMATION_ROADMAP.md

# TIS Future Automation Roadmap

This document lists possible future automation improvements. These are not implemented in Phase 3D.

## Git Hooks

Potential:

1. pre-commit check for changed docs without regenerated PDF/manifest,
1. warning when KIA-related docs are stale,
1. formatting checks for Markdown.

Considerations:

1. hooks must not block emergency work without clear override,
1. hooks must be documented for Windows/dev environments.

## CI Documentation Validation

Potential:

1. run PDF generator,
1. verify manifest is current,
1. verify required docs exist,
1. verify source hashes match,
1. fail pull requests when docs/PDF are stale.

## Automatic PDF Regeneration

Potential:

1. CI-generated PDF artifacts,
1. deployment-time regeneration,
1. explicit owner-only regenerate action.

Constraint:

1. automatic regeneration must not rewrite Markdown source docs.

## Automatic Manifest Generation

Potential:

1. generate manifest in CI,
1. compare manifest against committed PDF,
1. expose freshness status in Knowledge Center.

## Documentation Diff Reports

Potential:

1. summarize doc changes between commits,
1. show which docs changed after a feature,
1. include KIA summary in release notes.

## Stale Documentation Alerts

Potential:

1. Knowledge Center warning when manifest/source hashes differ,
1. CI alerts,
1. dashboard badge for owner users.

## Knowledge Center Search

Potential:

1. search docs, ADRs, module history, and roadmap from owner page,
1. filter by module,
1. link to protected document views.

## AI Semantic Search

Potential:

1. indexed semantic retrieval over KMS docs,
1. AI assistant prompt context packs,
1. module-specific context bundles.

Constraints:

1. preserve privacy,
1. do not expose internal docs publicly,
1. avoid unreviewed AI-generated source edits.

## Documentation Analytics

Potential:

1. track stale areas,
1. track frequently referenced docs,
1. identify docs missing module ownership.

## Release Documentation

Potential:

1. release notes generated from change history,
1. deployment checklist,
1. customer-facing change summaries,
1. internal technical release notes.

## Architecture Dashboards

Potential:

1. owner-facing architecture/decision dashboard,
1. ADR status summaries,
1. module health and documentation freshness.

## Priority Recommendation

Likely future order:

1. CI documentation validation.
2. Stale documentation alert in Knowledge Center.
3. Documentation diff report.
4. Owner-only explicit PDF regenerate action.
5. Search.
6. Semantic AI retrieval.

# docs/adr/0001-separate-nextjs-landing-website.md

# ADR 0001: Separate Next.js Landing Website

Status: Accepted

Date: 2026-06-26

## Context

TIS needs a public marketing website and a protected operational application. The public site has different performance, design, routing, and deployment needs than the FastAPI operational portal.

## Decision

Keep the public landing website in `tis-landing-website/` as a separate Next.js app. Keep the FastAPI app as the operational portal at `https://app.tisplatform.com`.

## Alternatives Considered

1. Use FastAPI/Jinja for both marketing and app pages.
1. Put marketing pages inside the operational app templates.
1. Use a static site generated outside the repository.

## Consequences

Positive:

1. Clear separation between marketing and operational concerns.
1. Landing page can evolve with a modern frontend stack.
1. Operational FastAPI app remains focused on authenticated workflows.

Tradeoffs:

1. Two runtimes must be understood.
1. Deployment and routing boundaries must be respected.

## Related Docs / Files

1. `tis-landing-website/`
1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/TIS_MASTER_CONTEXT.md`
1. `templates/landing.html`
1. `static/landing/landing.css`

# docs/adr/0002-separate-saas-identity-and-operational-users.md

# ADR 0002: Separate SaaS Identity And Operational Users

Status: Accepted

Date: 2026-06-26

## Context

TIS has SaaS account users who sign up, select plans, onboard organizations, and manage billing. It also has operational tenant users who work inside a provisioned school context. Platform owners and developers are separate platform identities.

## Decision

Keep SaaS account identity, platform identity, and operational tenant identity distinct.

## Alternatives Considered

1. Use one user table concept for all public signup, platform, and tenant users.
1. Automatically convert every SaaS signup into an operational tenant user.
1. Treat platform users as ordinary tenant administrators.

## Consequences

Positive:

1. Clearer security boundaries.
1. Safer tenant isolation.
1. Billing/onboarding can proceed before operational provisioning.

Tradeoffs:

1. Identity flows require careful documentation.
1. Developers must avoid mixing account, platform, and tenant assumptions.

## Related Docs / Files

1. `auth.py`
1. `models.py`
1. `saas/models.py`
1. `saas/service.py`
1. `saas/router.py`
1. `docs/TIS_MASTER_CONTEXT.md`

# docs/adr/0003-paddle-payment-architecture.md

# ADR 0003: Paddle Payment Architecture

Status: Accepted

Date: 2026-06-26

## Context

TIS requires subscription payments and billing visibility while preserving a clean boundary between academic operations and payment provider behavior.

## Decision

Use Paddle behind service/client boundaries in the `saas/` package. Keep payment, pricing, billing, and currency logic separate from academic modules.

## Alternatives Considered

1. Inline Paddle calls directly inside route handlers.
1. Store provider-specific behavior across operational modules.
1. Delay payment architecture until after provisioning.

## Consequences

Positive:

1. Provider integration is isolated.
1. Academic workflows stay independent of payment details.
1. Billing status can be reasoned about separately from onboarding and provisioning.

Tradeoffs:

1. Service boundaries must be maintained.
1. Webhook/payment state must be carefully tested.

## Related Docs / Files

1. `saas/paddle_client.py`
1. `saas/payment_service.py`
1. `saas/billing_service.py`
1. `saas/pricing_service.py`
1. `saas/currency_service.py`
1. `saas/router.py`
1. `docs/TIS_MASTER_CONTEXT.md`

# docs/adr/0004-webhook-only-payment-confirmation.md

# ADR 0004: Webhook-Only Payment Confirmation

Status: Accepted

Date: 2026-06-26

## Context

Checkout return pages can be reached by browser redirects, refreshes, or user navigation. They should not be treated as authoritative proof that payment succeeded.

## Decision

Treat provider webhook confirmation as the authoritative source for successful payment state. Checkout return pages may inform the user but should not independently confirm payment or trigger provisioning as if payment is verified.

## Alternatives Considered

1. Confirm payment based on checkout return route.
1. Allow manual UI confirmation from the SaaS account page.
1. Provision tenant records immediately after checkout redirect.

## Consequences

Positive:

1. Reduces risk of false-positive payment confirmation.
1. Aligns billing state with provider-confirmed events.
1. Supports delayed provisioning after verified payment.

Tradeoffs:

1. Users may need clear pending-payment status while webhook processing completes.
1. Webhook handling must be reliable and observable.

## Related Docs / Files

1. `saas/paddle_client.py`
1. `saas/payment_service.py`
1. `saas/billing_service.py`
1. `saas/router.py`
1. `docs/TIS_MASTER_CONTEXT.md`

# docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md

# ADR 0005: Delayed Tenant Provisioning After Verified Payment

Status: Accepted

Date: 2026-06-26

## Context

Public SaaS onboarding collects organization and branch information before an operational tenant exists. Provisioning creates operational school records and must be safe, reviewable, and recoverable.

## Decision

Delay tenant provisioning until payment is verified and the organization is ready for provisioning. Use platform owner provisioning workflows and jobs to create or update operational tenant structures.

## Alternatives Considered

1. Provision immediately after signup.
1. Provision immediately after checkout return.
1. Let public users create operational tenants directly.

## Consequences

Positive:

1. Protects operational tenant data.
1. Keeps pending SaaS onboarding separate from live school records.
1. Allows platform owner oversight and retry behavior.

Tradeoffs:

1. More state transitions exist between signup and operational access.
1. Provisioning status must be communicated clearly.

## Related Docs / Files

1. `saas/provisioning_service.py`
1. `saas/router.py`
1. `saas/service.py`
1. `templates/saas/admin_provisioning.html`
1. `templates/saas/admin_pending_organizations.html`
1. `docs/TIS_MASTER_CONTEXT.md`

# docs/adr/0006-documentation-as-source-knowledge-management-system.md

# ADR 0006: Documentation As Source / Knowledge Management System

Status: Accepted

Date: 2026-06-26

## Context

TIS is growing across product, SaaS, architecture, payment, provisioning, landing, and operational modules. Future humans and AI coding assistants need reliable context and change history.

## Decision

Use Markdown files under `docs/` as the source of truth. Generate the PDF booklet as a snapshot. Maintain change history, ADRs, module history, project state, master context, and AI project context as part of every meaningful development task.

## Alternatives Considered

1. Keep knowledge only in chat history.
1. Keep only a generated PDF.
1. Let the app rewrite docs automatically.
1. Store decisions only in commits.

## Consequences

Positive:

1. Project knowledge is reviewable and version-controlled.
1. Future Codex and ChatGPT conversations can onboard quickly.
1. Major decisions are preserved.

Tradeoffs:

1. Developers must maintain the KIA workflow.
1. Docs and PDF can become stale if the policy is ignored.

## Related Docs / Files

1. `docs/README.md`
1. `docs/DOCUMENTATION_UPDATE_POLICY.md`
1. `docs/CHANGE_HISTORY.md`
1. `docs/AI_PROJECT_CONTEXT.md`

```
scripts/generate_docs_pdf.py
static/docs/TIS_Project_Reference_Booklet.pdf
```

# docs/adr/0007-landing-page-visual-system-strategy.md

# ADR 0007: Landing Page Visual System Strategy

Status: Accepted

Date: 2026-06-26

## Context

The public landing page must communicate trust, product maturity, and academic operations value. It should not be accidentally changed during backend or operational app work.

## Decision

Treat the Next.js landing website and its visual system as a separate product surface. Keep marketing content references under `docs/marketing/`. Do not modify landing design, copy, or legacy FastAPI landing files unless explicitly approved.

## Alternatives Considered

1. Let backend tasks freely alter landing page presentation.
1. Continue using legacy FastAPI landing files as the public source.
1. Keep marketing strategy only in informal notes.

## Consequences

Positive:

1. Protects brand and public conversion strategy.
1. Keeps landing work deliberate and reviewable.
1. Reduces accidental coupling with operational app changes.

Tradeoffs:

1. Landing page changes need explicit planning.
1. Developers must check the source-of-truth docs before editing public website files.

## Related Docs / Files

1. `tis-landing-website/`
1. `docs/marketing/landing_page_source_of_truth.md`
1. `docs/marketing/tis_landing_page_master_content.md`
1. `docs/history/landing-page/README.md`

# docs/history/academic-calendar/README.md

## Academic Calendar History

This folder tracks meaningful changes to academic calendar workflows, event types, permissions, exports, and branch/year behavior.

Related files:

```
routers/academic_calendar.py
```

```
templates/academic_calendar.html
```

```
permission_registry.py
```

# docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md

# 2026-06-26 - KMS v3 Engineering Handbook

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Engineering Handbook

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only.

## Previous Documented State

The KMS had master context, project state, ADRs, module history, AI context, generated PDF, manifest, and Platform Owner Knowledge Center docs. It did not yet contain a complete engineering handbook for module ownership, repository architecture, and end-to-end flows.

## New Documented State

The KMS includes `docs/engineering/` with a module map, repository architecture guide, user/system flows, and developer onboarding index. The generated PDF now includes these docs as part of documentation version 3.0.

## Reason For Change

New human developers and future Codex/ChatGPT conversations need a practical engineering handbook, not only a bundle of source documents.

## User / Business Impact

Improves continuity, onboarding speed, technical review quality, and safer future implementation work.

## Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page, database, migration, route, or runtime behavior changed.

## Documentation Updated

1. `docs/engineering/README.md`
1. `docs/engineering/TIS_MODULE_MAP.md`
1. `docs/engineering/REPOSITORY_ARCHITECTURE.md`
1. `docs/engineering/USER_AND_SYSTEM_FLOWS.md`
1. `docs/TIS_MASTER_CONTEXT.md`
1. `docs/AI_PROJECT_CONTEXT.md`
1. `docs/PROJECT_STATE.md`
1. `docs/README.md`
1. `docs/CHANGE_HISTORY.md`

## PDF Regenerated

Yes

## Follow-Up Needed

Review PDF readability and handbook completeness before any Phase 3B work.

# docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md

# 2026-06-26 - KMS v3 Phase 3B Engineering Layers

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only.

## Previous Documented State

The engineering handbook included the module map, repository architecture guide, user/system flows, and onboarding index. It did not yet include a database architecture overview, development standards, UI/UX philosophy, or product roadmap.

## New Documented State

The engineering handbook includes Phase 3B layers:

1. database architecture overview,
1. development standards,
1. UI/UX design philosophy,
1. product roadmap,
1. stronger AI/human developer onboarding guidance.

## Reason For Change

New senior developers, Codex conversations, ChatGPT conversations, and technical reviewers need stronger system boundaries, design expectations, roadmap context, and development rules before changing TIS.

## User / Business Impact

Improves implementation safety, onboarding quality, technical review quality, and continuity across future development sessions.

## Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

## Documentation Updated

1. `docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md`
1. `docs/engineering/DEVELOPMENT_STANDARDS.md`
1. `docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md`

1 `docs/engineering/PRODUCT_ROADMAP.md`

1 core KMS index/context files

## PDF Regenerated

Yes

## Follow-Up Needed

Review handbook completeness and readability before any Phase 3C work.

# docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md

# 2026-06-26 - KMS v3 Phase 3C Governance And AI Traceability

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only.

## Previous Documented State

The engineering handbook included module maps, repository architecture, user/system flows, database architecture, development standards, UI/UX philosophy, and roadmap. It did not yet document rejected decisions, visual documentation standards, definitive AI guidance, governance, or decision traceability.

## New Documented State

The engineering handbook includes Phase 3C layers:

1. rejected architectural decisions,
1. visual documentation framework,
1. AI optimization guide,
1. project governance,
1. engineering decision traceability.

## Reason For Change

Future developers and AI assistants need to understand why TIS became what it is, not only its current structure.

## User / Business Impact

Improves continuity, reduces repeated architectural debates, improves AI readiness, and clarifies governance and review expectations.

## Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

## Documentation Updated

1. `docs/engineering/REJECTED_DECISIONS.md`
1. `docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md`
1. `docs/engineering/AI_OPTIMIZATION_GUIDE.md`

1 `docs/engineering/PROJECT_GOVERNANCE.md`

1 core KMS index/context files

## PDF Regenerated

Yes

## Follow-Up Needed

Future KMS improvements may add screenshots, diagrams, deployment runbooks, route inventories, test strategy docs, and deeper module guides. Do not implement those without a later approved phase.

# docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md

# 2026-06-26 - KMS v3 Phase 3D Lifecycle Foundation

Module: Engineering Handbook

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved as KMS v3.0 Phase 3D final phase.

## Previous Documented State

The engineering handbook documented module maps, repository architecture, flows, database architecture, standards, UI/UX philosophy, roadmap, rejected decisions, visual documentation, AI optimization, and governance. It did not yet define a full self-evolving lifecycle from task through deployment.

## New Documented State

The engineering handbook now includes:

1. knowledge lifecycle,
1. documentation automation guide,
1. formal KIA standard,
1. self-evolving workflow,
1. documentation dependency map,
1. AI coding workflow,
1. future automation roadmap.

This completes the KMS v1.0 lifecycle foundation.

## Reason For Change

Future implementation work needs a repeatable lifecycle that keeps KMS synchronized with software changes without automatic source-doc rewriting.

## User / Business Impact

Improves long-term governance, reviewer confidence, AI coding discipline, and continuity across future phases.

## Technical Impact

Documentation and PDF generation only. No SaaS flow, landing code, database model, migration, `tis.db`, app route, or runtime behavior changed.

## Documentation Updated

1. `docs/engineering/KNOWLEDGE_LIFECYCLE.md`

- `docs/engineering/DOCUMENTATION_AUTOMATION.md`
- `docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md`
- `docs/engineering/SELF_EVOLVING_WORKFLOW.md`
- `docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md`
- `docs/engineering/AI_CODING_WORKFLOW.md`
- `docs/engineering/FUTURE_AUTOMATION_ROADMAP.md`
- core KMS index/context files

## PDF Regenerated

Yes

## Follow-Up Needed

Review the final handbook for readability. Future improvements should be handled as new approved KMS phases.

# docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md

# 2026-06-27 - Production Memory Stability Guardrails

Module: Engineering Handbook and operational app stability

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-27 - Added Production Memory Stability Guardrails

Related ADRs: None. This is a standards and operational guardrail update, not a new architecture decision.

Reviewer/approval notes: Prepared after a Render restart/502 investigation. Local changes only; no commit, push, migration, database change, or deployment was performed.

## Previous Documented State

The KMS required scoped changes, tenant isolation, validation, and documentation updates, but it did not explicitly define a production memory budget or prohibit common memory hazards such as:

1. full large-dataset parsing for normal picker requests,
1. duplicate template rendering in production routes,
1. warning-level diagnostic stage logs during normal traffic,
1. unbounded global caches,
1. startup-heavy work without memory review.

## New Documented State

`docs/engineering/DEVELOPMENT_STANDARDS.md` now includes a mandatory Production Memory and Render Stability section.

The standards require future work to:

1. treat 512 MB Render memory as a hard production budget,
1. avoid full large-dataset loads when scoped or streaming lookup is available,
1. avoid duplicate template renders in production request paths,
1. keep diagnostics opt-in through explicit environment flags,
1. filter data before materializing result sets,
1. keep caches bounded and justified,
1. include memory/performance smoke checks for memory-sensitive changes.

## Reason For Change

The 2026-06-27 production investigation found avoidable memory pressure risks after deployment:

1. `/observations/` had diagnostic logging and extra template pre-renders in the normal route path.
1. Global location lookup could parse a 47 MB country/state/city reference dataset into a complete in-memory index for simple picker API calls.

These patterns can trigger process restarts and temporary 502 responses on constrained production instances.

## User / Business Impact

The rule reduces the risk that normal navigation causes Render restarts, temporary 502s, or user-visible instability. It also creates a clear review standard for future SaaS onboarding, dashboard, reports, observations, and large-data changes.

## Technical Impact

Future changes that add broad data loading, reference datasets, exports, diagnostics, or startup work must include memory-conscious design and validation. The local stabilization patch also reduces current risk by gating observation diagnostics and changing location lookup to scoped loading.

## Documentation Updated

1 `docs/engineering/DEVELOPMENT_STANDARDS.md`

1 `docs/PROJECT_STATE.md`

1 `docs/AI_PROJECT_CONTEXT.md`

1 `docs/TIS_MASTER_CONTEXT.md`

1 `docs/CHANGE_HISTORY.md`

## PDF Regenerated

Yes

## Follow-Up Needed

Review, commit, and deploy the stabilization changes when approved. After deployment, monitor Render memory, restart count, warning logs, and 502 frequency.

# docs/history/engineering-handbook/README.md

## Engineering Handbook History

This folder tracks meaningful changes to the TIS Engineering Handbook, including module maps, repository architecture, user/system flows, and onboarding guidance for humans and AI coding conversations.

Related files:

```
docs/engineering/README.md
docs/engineering/TIS_MODULE_MAP.md
docs/engineering/REPOSITORY_ARCHITECTURE.md
docs/engineering/USER_AND_SYSTEM_FLOWS.md
```

# docs/history/landing-page/README.md

## Landing Page History

This folder tracks meaningful changes to the public landing page, marketing positioning, visual system strategy, and source-of-truth boundaries.

Related docs:

- `docs/adr/0001-separate-nextjs-landing-website.md`
- `docs/adr/0007-landing-page-visual-system-strategy.md`
- `docs/marketing/landing_page_source_of_truth.md`
- `docs/marketing/tis_landing_page_master_content.md`

# docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

# 2026-06-26 - Platform Owner Knowledge Center

Module: Platform Knowledge Center

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Added Platform Owner Knowledge Center

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 2C. Regenerate button, public access, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

## Previous Documented State

TIS had KMS Markdown source files, ADRs, module history, a generated PDF booklet, and a manifest, but there was no protected in-app owner utility for viewing KMS status or accessing the booklet.

## New Documented State

TIS has a read-only Platform Owner Knowledge Center that shows KMS health, manifest metadata, PDF freshness, source document status, recent change-history entries, ADRs, module history areas, and the Knowledge Impact Assessment checklist.

## Reason For Change

Platform owners need a protected internal view of KMS status and a safe way to view/download the generated PDF without linking directly to static public paths.

## User / Business Impact

Platform owners can verify whether the KMS snapshot is current and access the reference booklet from inside the app.

## Technical Impact

Adds read-only KMS service helpers, owner-protected FastAPI routes, one app-shell template, and an owner-only Platform Console card. No SaaS, database, migration, billing, provisioning, or landing page behavior changes.

## Documentation Updated

- `docs/TIS_MASTER_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/CHANGE_HISTORY.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/history/platform-knowledge/README.md`
- `docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md`

## PDF Regenerated

Yes

## Follow-Up Needed

Consider a future explicit owner-only regenerate action after review. The app must not silently rewrite Markdown source docs.

# docs/history/platform-knowledge/README.md

# Platform Knowledge Center History

This folder tracks meaningful changes to the owner-only TIS Knowledge Center, protected documentation PDF access, KMS metadata display, freshness detection, and knowledge health reporting.

Related files:

- `knowledge_service.py`
- `templates/platform_knowledge_center.html`
- `templates/platform_console.html`
- `main.py`

# docs/history/provisioning/2026-06-26-kms-foundation.md

# 2026-06-26 - KMS Foundation

Module: Documentation and knowledge management

Related change-history entry: `docs/CHANGE_HISTORY.md` - 2026-06-26 - Established Knowledge Management System Foundation

Related ADRs: `docs/adr/0006-documentation-as-source-knowledge-management-system.md`

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only.

## Previous Documented State

TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal KMS policy, ADR structure, module history foundation, or AI onboarding context.

## New Documented State

TIS now has a KMS foundation with policy, change history, ADRs, module history folders, and AI project context. The PDF generator includes these source docs and produces manifest metadata.

## Reason For Change

Future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and reviewers need durable project context and history.

## User / Business Impact

Improves continuity, reduces repeated rediscovery, and makes project decisions easier to review.

## Technical Impact

No runtime app behavior changed. This is documentation and PDF generation foundation only.

## Documentation Updated

- `docs/CHANGE_HISTORY.md`
- `docs/DOCUMENTATION_UPDATE_POLICY.md`
- `docs/AI_PROJECT_CONTEXT.md`
- `docs/adr/`
- `docs/history/`
- `docs/TIS_MASTER_CONTEXT.md`
- `docs/PROJECT_STATE.md`
- `docs/README.md`

## PDF Regenerated

Yes

## Follow-Up Needed

Phase 2C can later add a protected Platform Owner Knowledge Center after review.

# docs/history/provisioning/README.md

# Provisioning History

This folder tracks meaningful changes to pending organizations, provisioning jobs, verified-payment readiness, retry behavior, and platform owner provisioning oversight.

Related docs:

```
docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
```

```
docs/TIS_MASTER_CONTEXT.md
```

Related files:

```
saas/provisioning_service.py
```

```
saas/router.py
```

```
templates/saas/admin_provisioning.html
```

# docs/history/README.md

# TIS Module History

Module history stores deeper area-specific change records. It complements `docs/CHANGE_HISTORY.md`, which remains the chronological summary.

Use module history when a meaningful area changes and the previous documented state should be preserved.

## Module Areas

```
1    subscriptions/
1    landing-page/
1    academic-calendar/
1    workforce-planning/
1    saas-onboarding/
1    provisioning/
```

## Entry Template

```
# YYYY-MM-DD - Change Title

Module:
Related change-history entry:
Related ADRs:
Reviewer/approval notes:

## Previous Documented State

## New Documented State

## Reason For Change

## User / Business Impact

## Technical Impact

## Documentation Updated

## PDF Regenerated

## Follow-Up Needed
```

# docs/history/saas-onboarding/README.md

## SaaS Onboarding History

This folder tracks meaningful changes to signup, login, account, organization onboarding, contacts, branches, academic setup, review, and account self-service.

Related files:

```
saas/router.py
saas/service.py
templates/saas/
docs/adr/0002-separate-saas-identity-and-operational-users.md
```

# docs/history/subscriptions/README.md

## Subscription History

This folder tracks meaningful changes to TIS subscription plans, pricing, billing status, payment behavior, checkout assumptions, and provider boundaries.

Related docs:

```
docs/adr/0003-paddle-payment-architecture.md
```

```
docs/adr/0004-webhook-only-payment-confirmation.md
```

```
docs/TIS_MASTER_CONTEXT.md
```

# docs/history/workforce-planning/README.md

# Workforce Planning History

This folder tracks meaningful changes to teacher information, workload planning, staffing gaps, planning sections, timetable relationships, and related reports.

Related files:

```
routers/teachers.py
routers/planning.py
routers/timetable.py
teacher_capacity.py
timetable_logic.py
```

# docs/marketing/landing_page_source_of_truth.md

# Landing Page Source of Truth

The official source of truth for the public TIS landing website is:

`tis-landing-website/`

## Public Website

- **Domain:** https://tisplatform.com
- **Runtime:** Next.js / Node
- **Local testing URL:** http://localhost:3000

All future marketing landing page changes must be made inside `tis-landing-website/`.

## Application Portal

The TIS application portal remains separate from the public landing website:

- **Domain:** https://app.tisplatform.com
- **Runtime:** FastAPI / Python with PostgreSQL

## Legacy Landing Page Files

The former FastAPI/Jinja landing page files are now legacy:

- `templates/landing.html`
- `static/landing/landing.css`

Codex and other developers must not modify these legacy files unless explicitly instructed.

# docs/marketing/tis_landing_page_master_content.md

TIS Landing Page Master Content Approved Marketing Foundation

1 Main Positioning

TIS is a developing SaaS academic operations platform designed to connect school leadership, supervisors, teachers, branches, calendars, observations, staffing data, and decision-making inside one integrated system. The platform helps schools organize teacher information, analyze teaching loads, identify staffing gaps, coordinate shared academic calendars, manage structured observation cycles, support supervisor-teacher communication, and build clearer visibility across academic operations. As TIS continues to grow, advanced AI-powered features will be introduced through subscription-based plans, including curriculum-driven assessment generation, academic analytics, planning support, and intelligent recommendations based on verified school data.

1 Hero Section

Hero Badge Built for Schools Moving Beyond Spreadsheets Main Headline When Academic Decisions Are Scattered, Schools Lose Time, Clarity, and Control. Subheadline TIS brings teacher data, workloads, staffing gaps, academic calendars, observations, and leadership decisions into one connected SaaS platform - helping schools see what is happening, what is missing, and what needs action. CTA Buttons Request Early Access See How TIS Works Small Supporting Line A developing academic operations platform with advanced AI capabilities planned through subscription-based growth.

1 Problem Section

Section Label The Problem Main Heading Schools Are Not Short of Data. They Are Drowning in Disconnected Files. Section Text In many schools and educational organizations, every academic process becomes another spreadsheet. Teacher loads, subject coverage, observations, calendars, reports, staffing needs, and follow-up tasks are often managed in separate files, created by different people, in different formats. Over time, this creates a hidden operational burden. Staff members spend more time updating files than understanding the data. Leaders receive information late, inconsistently, or from multiple disconnected sources. In multi-branch organizations, the problem becomes even larger: every branch may work differently, every file may tell a different story, and every decision requires extra checking. Problem Cards

1 Too Many Spreadsheets

Academic work is scattered across different files, formats, and personal tracking systems.

1 Inconsistent Data

Each person or branch may structure information differently, making comparison and reporting difficult.

1 Heavy Staff Workload

Teachers, supervisors, and coordinators carry the burden of repeatedly entering and updating data.

1 Delayed Visibility

Leaders often discover staffing gaps, missing coverage, or academic issues after the problem has already grown.

1 Disconnected Decisions

Calendars, observations, staffing, reports, and academic planning are managed separately instead of working as one connected system.

1 Solution Section

Section Label The Solution Main Heading One Connected Platform to Bring Academic Operations Back Under Control. Section Text TIS brings the scattered pieces of school operations into one structured academic ecosystem. Instead of chasing separate spreadsheets, comparing different file formats, and waiting for updates from multiple sources, school leaders can work from a clearer, more connected view of what is happening across their school or organization. Teacher data, workloads, subject coverage, academic calendars, observations, supervision follow-up, branch structures, and leadership decisions become part of one system. This allows schools to reduce manual confusion, improve visibility, standardize academic processes, and make decisions with greater confidence. For multi-branch organizations, TIS helps create consistency across campuses while still allowing each branch to manage its own academic operations within a secure, organized, and tenant-isolated platform. Solution Cards

1 One Source of Academic Truth

Bring teacher data, workloads, observations, calendars, and staffing information into one connected system.

1 Clearer Leadership Visibility

Help principals, academic leaders, and supervisors see what is happening, what is missing, and what requires action.

1 Standardized Academic Processes

Reduce personal spreadsheet formats and create a more consistent way to manage academic operations across branches.

1 Smarter Staffing Decisions

Identify teaching load issues, uncovered hours, subject gaps, and staffing needs before they become larger problems.

1 Connected School Communication

Support better coordination between leadership, supervisors, teachers, and branches through shared academic workflows.

1 Ready for Future Intelligence

Build the structured data foundation needed for advanced AI-powered features as the platform continues to grow.

1 Core Platform Capabilities

Section Label Core Platform Capabilities Main Heading Everything Your Academic Operation Needs - Connected in One Platform. Section Intro TIS is designed to bring the most important academic operations into one structured SaaS environment. From teacher planning and branch management to observations, calendars, dashboards, and future AI-powered intelligence, the platform helps schools move from scattered work to connected academic control. Capability Cards

1 Academic Workforce Planning

Plan teacher assignments, analyze weekly loads, identify uncovered teaching hours, and understand staffing needs with greater clarity.

1 Academic Calendars & Shared Coordination

Coordinate academic events, timelines, school priorities, and branch-level planning through shared calendars that keep teams aligned.

1 Observation & Supervision Workflows

Manage structured teacher observations, supervisor feedback, shared records, and signing workflows inside one organized process.

1 Multi-Branch Organization Management

Support school groups, campuses, branches, users, roles, and tenant-isolated structures from one scalable SaaS platform.

1 Dashboards & Decision Visibility

Give leaders a clearer view of coverage, staffing gaps, teacher utilization, subject sufficiency, and academic follow-up needs.

1 Future AI-Powered Intelligence

Prepare for advanced subscription-based AI capabilities, including curriculum-driven assessment generation, answer keys, planning support, analytics, and intelligent recommendations based on verified academic data.

1 Teacher Workforce Planning Engine

Section Label Teacher Workforce Planning Engine Main Heading Know Exactly Who Is Teaching What - and Where Support Is Needed. Section Text One of the strongest current capabilities of TIS is its ability to help schools understand their teaching workforce with clarity. Instead of manually calculating teacher loads, subject coverage, and staffing needs across separate files, TIS brings this information into one structured planning engine. School leaders can track teacher qualifications, majors, subject specialties, weekly teaching loads, assigned subjects, uncovered hours, and additional staffing requirements. This helps academic teams identify gaps earlier, use existing teachers more effectively, and make staffing decisions based on clear data rather than assumptions. For schools and multi-branch organizations, this creates a more reliable way to plan academic coverage, compare staffing needs, and understand whether each subject, grade, and branch has the right teaching support. Key Capabilities

1 Teacher Load Visibility

See teacher workloads clearly and understand whether teaching hours are balanced, underused, or overloaded.

1 Subject Coverage Analysis

Identify which subjects are fully covered and which still have uncovered teaching hours.

1 Staffing Gap Detection

Detect shortages early before they affect schedules, instruction, or academic quality.

1 Qualification-Based Planning

Use teacher degrees, majors, and subject specialties to support smarter assignment decisions.

1 New Teacher Requirement Insights

Understand how many additional teachers may be needed based on uncovered hours and school workload requirements.

1 Multi-Branch Workforce Clarity

Support better staffing visibility across branches, campuses, and larger educational organizations.

1 Observation & Supervision Workflows

Section Label Observation & Supervision Workflows Main Heading Turn Teacher Observation into a Clear, Shared, and Accountable Process. Section Text TIS is designed to help schools move teacher observation away from scattered forms, disconnected files, and informal follow-up. Observation records, supervisor feedback, teacher responses, signatures, and approval steps can become part of one structured academic workflow. This gives supervisors a clearer way to document classroom visits, teachers a more transparent way to receive and respond to feedback, and school leaders a stronger view of supervision quality across departments and branches. As the platform continues to grow, advanced AI-assisted supervision features are planned to support supervisors by suggesting improvement plans based on observation notes, evaluation results, teacher performance evidence, and school-approved criteria. Key Capabilities

1 Structured Observation Records

Create organized observation forms and records that follow a clear school-approved process.

1 Shared Teacher-Supervisor Visibility

Allow teachers and supervisors to access relevant observation records, feedback, and follow-up actions.

1 Signing and Approval Workflow

Support formal documentation through review, acknowledgement, signing, and approval steps.

1 Supervision Follow-Up

Track feedback, recommendations, and improvement actions after classroom observations.

1 AI-Suggested Improvement Plans

Future AI-assisted tools may help supervisors generate suggested teacher improvement plans based on observation notes, evaluation data, and approved academic criteria.

1 Leadership Oversight

Give academic leaders better visibility over observation activity, supervision quality, and follow-up completion.

1 Reduced Observation Paperwork

Replace scattered observation files with a more consistent, traceable, and connected supervision system.

1 Academic Calendars & Shared Coordination

Section Label Academic Calendars & Shared Coordination Main Heading Turn the Academic Calendar into a Shared Planning Engine. Section Text TIS is designed to help schools move beyond static calendars and disconnected event planning. Academic leaders, supervisors, teachers, and branches can work from a shared academic calendar connected to the selected academic year, helping the whole school coordinate priorities, events, activities, observations, deadlines, and academic follow-up. Future calendar capabilities can support default worldwide and international academic events, allowing schools to plan meaningful activities around important global occasions. This gives international, national, private, and public schools a stronger way to connect school events with classroom learning, curriculum objectives, student engagement, and whole-school awareness. As TIS continues to grow,

AI-assisted calendar planning can help schools generate suggested activities for selected events, including classroom activities, outdoor activities, entertainment activities, curriculum-integrated tasks, and lesson-based ideas. The calendar vision also includes support for Sustainable Development Goals awareness planning, helping schools design year-round activities connected to SDGs, global citizenship, and curriculum integration. Key Capabilities

1 Academic-Year-Based Calendar

Build and manage school calendars according to the selected academic year, such as 2026-2027.

1 Shared School Coordination

Connect leaders, supervisors, teachers, and branches through one shared academic planning calendar.

1 International Event Awareness

Support worldwide and international academic events that schools can use for activities, campaigns, and curriculum connections.

1 AI-Suggested Activity Planning

Future AI-assisted tools may suggest activity plans based on selected events, school needs, grade levels, and curriculum links.

1 Curriculum-Integrated Events

Help schools connect calendar events with classroom learning, lesson planning, and academic objectives.

1 SDG Awareness Planning

Support year-round planning for Sustainable Development Goals awareness, student engagement, and global citizenship activities.

1 Branch-Level Visibility

Give school groups and multi-branch organizations a clearer view of academic events, priorities, and planning across campuses.

1 Multi-Branch Organization Management

Section Label Multi-Branch Organization Management Main Heading Built for Schools That Operate Across Branches, Campuses, and Teams. Section Text TIS is designed to support schools and educational organizations that need structure beyond a single location. Whether an organization manages one school, several branches, multiple campuses, or different academic pathways, TIS provides a scalable environment where users, roles, branches, and academic operations can be organized with greater clarity. School leadership and organization administrators can manage branches, users, permissions, branding, academic structures, and operational workflows inside a tenant-isolated SaaS environment. This helps every branch work within a shared organizational framework while still maintaining its own operational identity and academic needs. For educational groups, TIS can support stronger management-level visibility across branches. Leaders can monitor branch progress, compare completion levels, identify delayed tasks, and recognize where follow-up is needed across areas such as curriculum planning, exams, observations, calendars, reporting, and academic operations. Key Capabilities

1 Organization and Branch Structure

Manage organizations, schools, branches, campuses, and academic pathways inside one scalable platform.

1 Role-Based User Management

Assign users according to their responsibilities, such as organization administrator, branch leader, academic leader, supervisor, teacher, or future custom roles.

1 Tenant-Isolated Environment

Keep each organization's data, users, branding, and academic operations securely separated.

1 Branch Progress Intelligence

Give management-level users a clearer view of each branch's completion, progress, delays, and operational follow-up needs.

1 Branch Comparison Indicators

Compare branch performance across connected academic areas such as curriculum tasks, exams, observations, calendars, planning, and reports.

1 Custom Branding by Organization

Support organization and branch branding, including logos, visual identity, and school-specific presentation.

1 Scalable SaaS Structure

Prepare schools and educational groups for growth, future modules, subscription plans, and advanced AI-powered capabilities.

1 Dashboards & Decision Visibility

Section Label Dashboards & Decision Visibility Main Heading See Academic Progress, Risks, and Priorities Before They Become Problems. Section Text TIS dashboards are designed to give school leaders more than static reports. They provide a connected view of academic operations, helping leadership understand what has been completed, what is delayed, what needs follow-up, and where intervention may be required. As the platform continues to grow, dashboards can reflect progress across annual plans, weekly plans, curriculum coverage, assessment blueprints, quizzes, exams, academic calendar deadlines, teacher observations, supervision follow-up, staffing gaps, and branch-level performance. For branch management, dashboards can show the academic health of one school or campus. For organization-level leadership, dashboards can compare branches, highlight delayed tasks, identify performance gaps, and support faster, evidence-based decisions. Key Capabilities

1 Planning Completion Indicators

Track progress for annual plans, weekly plans, lesson planning, curriculum coverage, and completed academic tasks.

1 Assessment Progress Visibility

Monitor assessment blueprints, quizzes, exams, term-based assessment requirements, and calendar-linked due dates.

1 Observation & Supervision Insights

See observation progress, teacher follow-up status, supervisor actions, and areas requiring improvement plans.

1 Academic Calendar Integration

Connect events, deadlines, assessments, activities, and required tasks to dashboard indicators.

1 Teacher & Academic Performance Indicators

Support leadership with visual indicators related to teacher follow-up, academic progress, and school-level performance trends.

1 Branch and Organization Dashboards

Provide branch-level dashboards for school management and organization-level dashboards for comparing progress across multiple campuses.

1 Risk and Delay Alerts

Highlight delayed tasks, incomplete requirements, missing academic evidence, and branches or departments that need attention. Supporting Line: From reports to real-time academic visibility.

1 Future AI-Powered Intelligence

Section Label Future AI-Powered Intelligence Main Heading AI That Works Inside the Academic Workflow - Not Outside It. Section Text TIS is being developed with a future AI vision that goes beyond simple content generation. Instead of using AI as a separate tool, future AI-powered capabilities are planned to work inside the platform's academic workflows, supporting schools through the data already stored, reviewed, and approved in TIS. This future intelligence layer can support academic calendars, observation follow-up, curriculum planning, teacher support, assessment generation, dashboards, branch progress indicators, and leadership decision-making. At the center of this vision is a curriculum-to-classroom workflow. Uploaded curriculum data, standards, learning outcomes, annual plans, weekly plans, lesson plans, and confirmed instructional coverage can become the foundation for guided planning, AI-assisted recommendations, assessment generation, answer keys, and academic analytics. Future AI capabilities will be introduced progressively through subscription-based plans, allowing schools to access more advanced intelligence features as their needs grow. Key Capabilities

1 Curriculum-to-Classroom AI Support

Assist teachers and academic leaders in moving from uploaded curriculum to annual plans, weekly plans, lesson plans, and verified coverage.

1 AI-Assisted Assessment Generation

Support future generation of exams, quizzes, assessment blueprints, and answer keys based on approved curriculum data and confirmed instructional coverage.

1 AI-Supported Observation Follow-Up

Help supervisors generate suggested teacher improvement plans based on observation notes, evaluation results, and school-approved criteria.

1 AI-Powered Calendar Planning

Suggest activities for international events, SDG awareness, curriculum links, classroom activities, and school-wide academic events.

1 Academic Analytics and Recommendations

Provide future intelligent insights related to planning progress, teacher follow-up, assessment readiness, curriculum coverage, and branch performance.

1 Subscription-Based AI Growth

Introduce advanced AI features progressively through higher subscription plans, giving schools flexibility to grow into more powerful capabilities.

1 From Curriculum to Classroom - With Guided AI Support

Section Label From Curriculum to Classroom Main Heading Support Teachers Without Adding Another Burden. Section Text TIS is being developed to support teachers, not overload them. Instead of asking teachers to build annual plans, weekly plans, lesson plans, and curriculum tracking documents from scratch, future AI-assisted planning tools will help guide them through structured steps using approved templates, uploaded curriculum data, learning outcomes, standards, objectives, and school-specific requirements. Teachers will be able to review, adjust, complete, and submit their plans through the system. Academic leaders can then review and approve the work, turning planning documents into reliable, official academic data inside TIS. This helps reduce teacher workload, improve planning consistency, and give school leadership clearer visibility over what has been planned, approved, taught, and covered.

1 Subscription Plans Preview

Section Label Subscription Plans Main Heading Choose the Level of Academic Intelligence Your School Needs. Section Text TIS is being developed as a subscription-based SaaS platform, allowing schools and educational organizations to start with essential academic operations and grow into more advanced capabilities over time. As the platform expands, subscription plans can support different levels of access, from core teacher and academic operations to advanced dashboards, multi-branch management, supervision workflows, and future AI-powered intelligence. Plan Cards

1 Core

For schools that need a structured way to organize teacher data, academic operations, basic planning visibility, and essential workflows.

1 Professional

For schools that need stronger dashboards, shared academic calendars, observation workflows, staffing visibility, and improved coordination across teams.

1 Enterprise AI

For school groups and larger organizations that need multi-branch visibility, advanced customization, branch progress intelligence, AI-assisted planning, assessment generation, analytics, and strategic decision support. Supporting Note AI-powered capabilities will be introduced progressively and may vary according to subscription level, school needs, and platform development stage. CTA Request Early Access Request Pricing

1 Final Call-to-Action

Section Label Start Your TIS Journey Main Heading Ready to Bring Clarity, Control, and Intelligence to Your Academic Operations? Section Text TIS is being built for schools and educational organizations that want to move beyond scattered files, disconnected workflows, and delayed decisions. Start exploring how one connected academic operations platform can support your leadership team, supervisors, teachers, branches, and future AI-powered growth. CTA Buttons Request Early Access Book a Demo Supporting Line TIS is currently in active development, with selected features and advanced AI capabilities planned for progressive release through subscription-based plans.

# docs/location-data-roadmap.md

# TIS Location Data Roadmap

## Phase 1: Global Form Usability

Status: implemented in the current working tree.

Phase 1 intentionally keeps location data attached directly to an organization or branch. It does not create shared locality records.

Included:

1. Controlled country selection from the local dataset.
1. Controlled region/state/province selection when dataset coverage exists.
1. Dataset-backed city/locality selection.
1. `Other / manual entry` fallback for missing regions and cities.
1. Optional district and neighborhood fields, stored separately from city.
1. Local cached APIs; no external location API dependency at runtime.
1. Existing text location fields remain compatible with legacy Saudi data.

Manual region and city values are record-level values. They are not published to other tenants and are not added to the imported dataset.

## Deferred Architecture

The following phases are roadmap items only. They are not part of Phase 1 and must not be inferred from the current text-based storage model.

### Locality Registry

Create stable TIS locality identifiers and canonical locality records that are independent from vendor dataset identifiers. Organizations and branches would eventually reference these records while retaining display-name snapshots for audit and migration compatibility.

### Locality Aliases

Support native names, translated names, transliterations, spelling variants, and historical names. Alias matching should be region-aware and must not merge places solely because their normalized names match.

### GeoNames and Official Dataset Enrichment

Build an offline enrichment process using GeoNames country dumps and, where available, official national gazetteers. Source priority, licensing, attribution, place classification, and duplicate resolution must be defined before combining records.

### Tenant-Private City Additions

Allow an authorized tenant user to suggest a missing locality. New records should initially be visible only to that tenant and must be constrained to the tenant's organization scope.

### Platform Moderation

Provide a Platform Owner moderation queue for reviewing proposed localities, potential duplicates, coordinates, place type, aliases, and source evidence.

### City Verification and Promotion

Support explicit states such as tenant-private, pending, verified, deprecated, and merged. Promotion to the global catalog must be an audited platform action, never an automatic consequence of a tenant entering free text.

### Dataset Refresh and ETL Workflow

Introduce versioned, offline imports with source checksums, staging tables, coverage metrics, diff review, attribution records, and rollback support. Referenced localities should be deprecated or merged rather than deleted.

## Guardrails for Future Work

1. Preserve tenant isolation for all tenant-created locality data.
1. Keep stable internal IDs separate from external source IDs.
1. Retain source and license provenance for every imported record.
1. Do not call public geocoding services from normal page requests.
1. Do not modify the vendor JSON to store tenant-entered values.
1. Migrate existing organization and branch text values without data loss.